

Programa de formación
MACHINE LEARNING AND DATA SCIENCE MLDS

Facultad de INGENIERÍA

UNIVERSIDAD NACIONAL DE COLOMBIA

Módulo 5

Deep learning: Introducción al Aprendizaje Profundo con Python

Unidad 3

Redes neuronales Convolucionales

Fabio Augusto González Osorio, PhD

Facultad de INGENIERÍA

UNIVERSIDAD NACIONAL DE COLOMBIA

Bienvenida

Oscar Julián Perdomo Charry, PhD

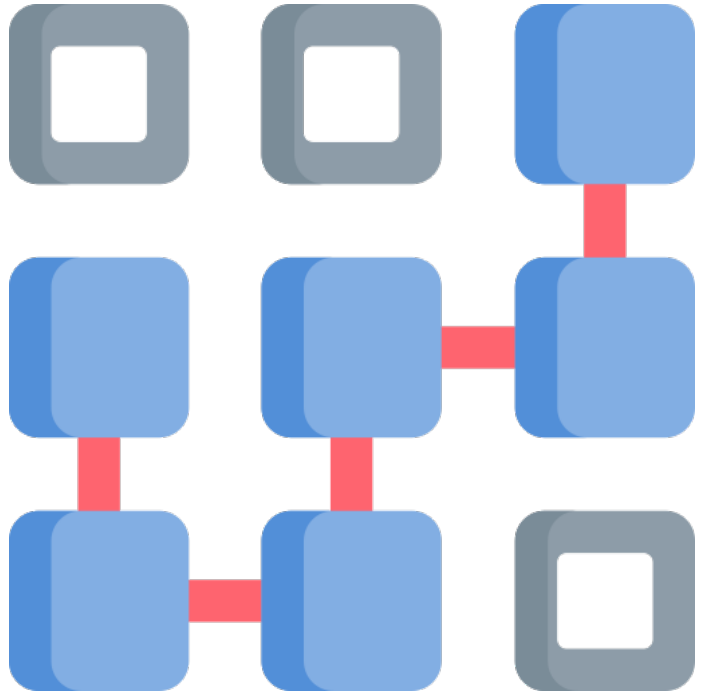
ojperdomoc@unal.edu.co

Departamento de Ingeniería Eléctrica y Electrónica
Facultad de Ingeniería
Universidad Nacional de Colombia
Sede Bogotá

UNIVERSIDAD NACIONAL DE COLOMBIA

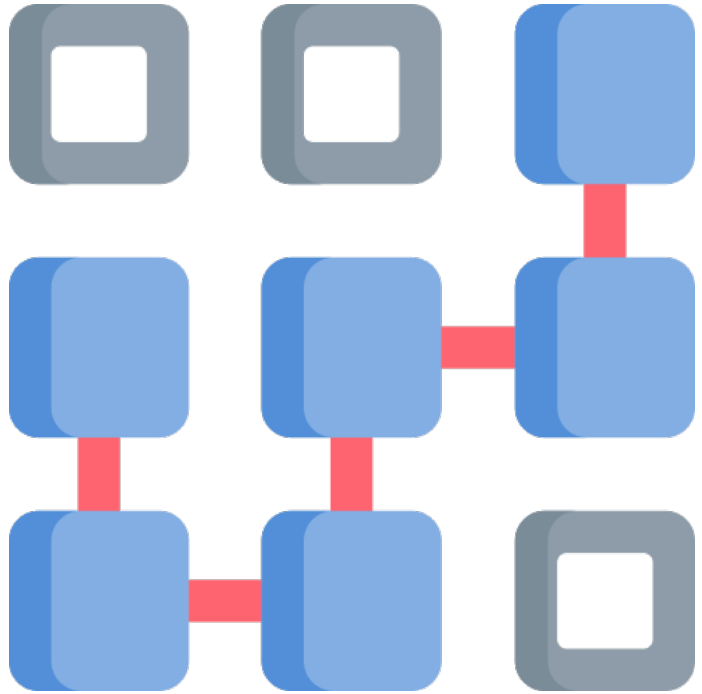
Agenda

- Introducción a Redes Neuronales Convolucionales.
- Capas Convolucionales y de Pooling.
- Data Augmentation.
- Transfer Learning
- Fine Tuning.
- Transfer Learning y Fine Tuning en Keras

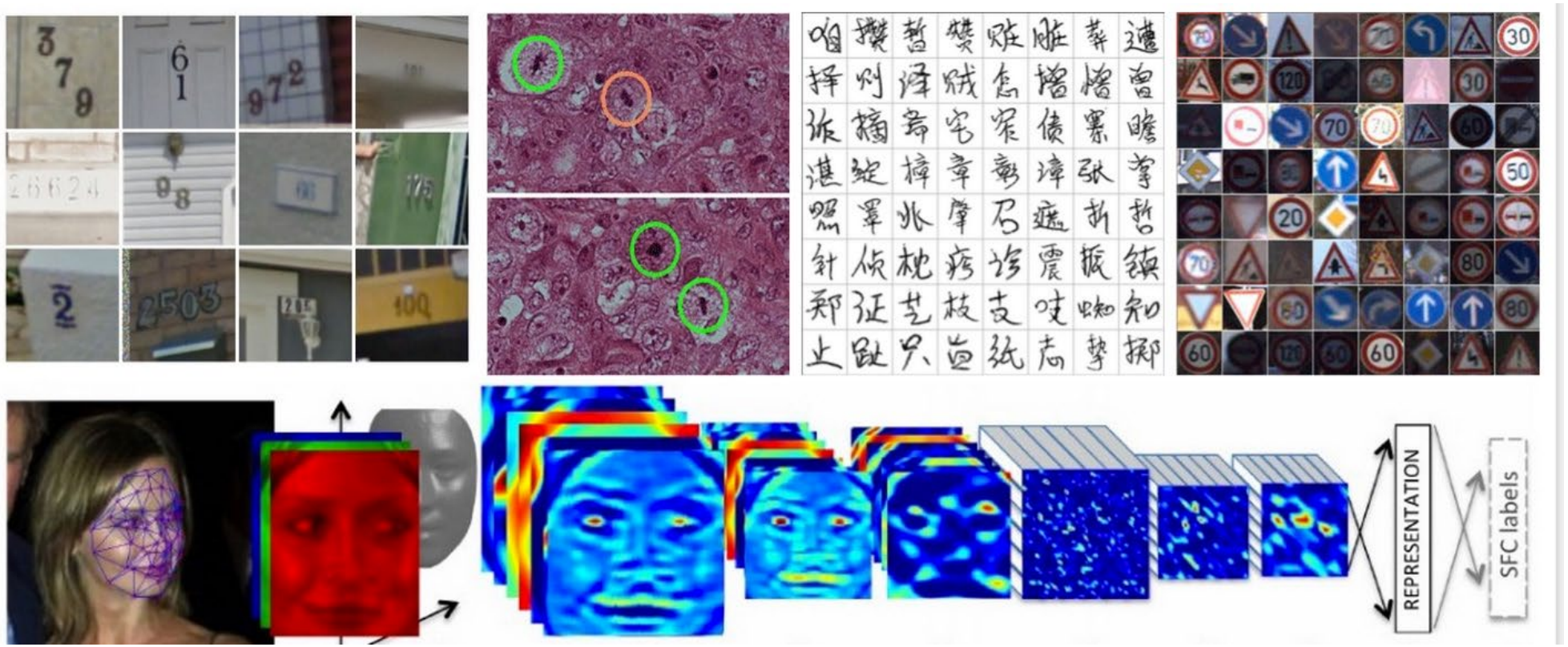


Agenda

- Introducción a Redes Neuronales Convolucionales.
- Capas Convolucionales y de Pooling.
- Data Augmentation.
- Transfer Learning
- Fine Tuning.
- Transfer Learning y Fine Tuning en Keras

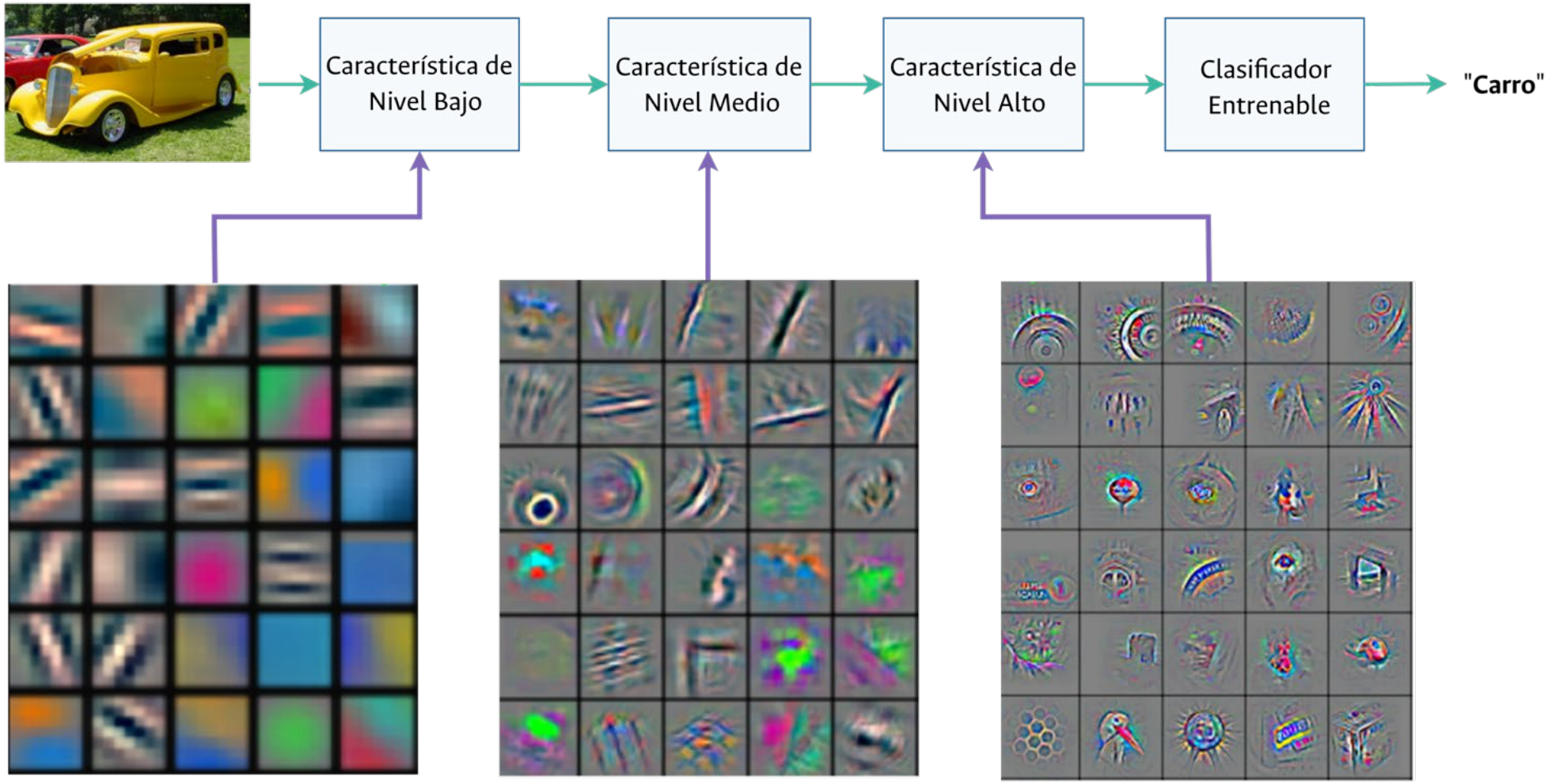


1 Introducción a redes Neuronales Convolucionales



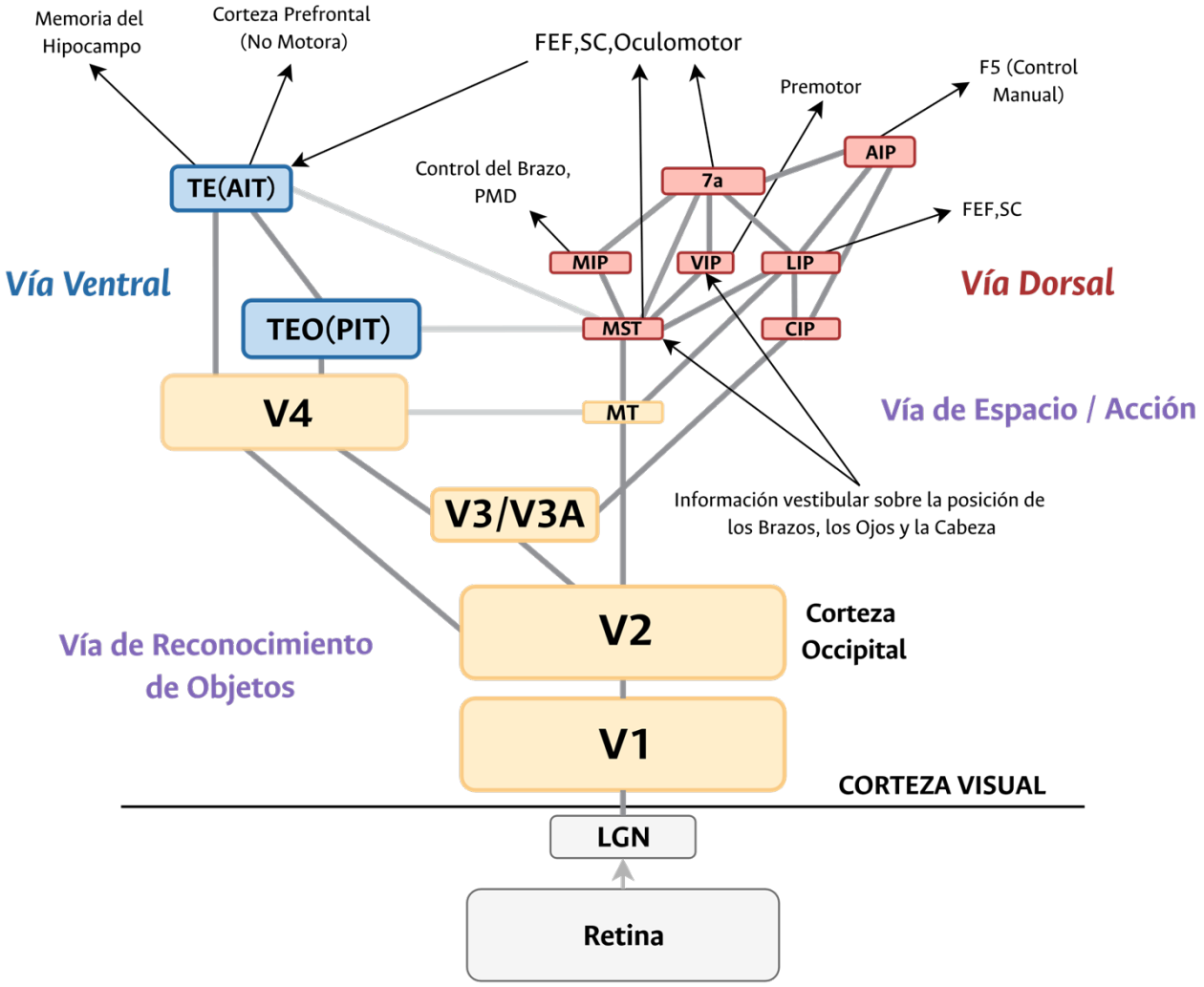
Introducción a redes Neuronales Convolucionales

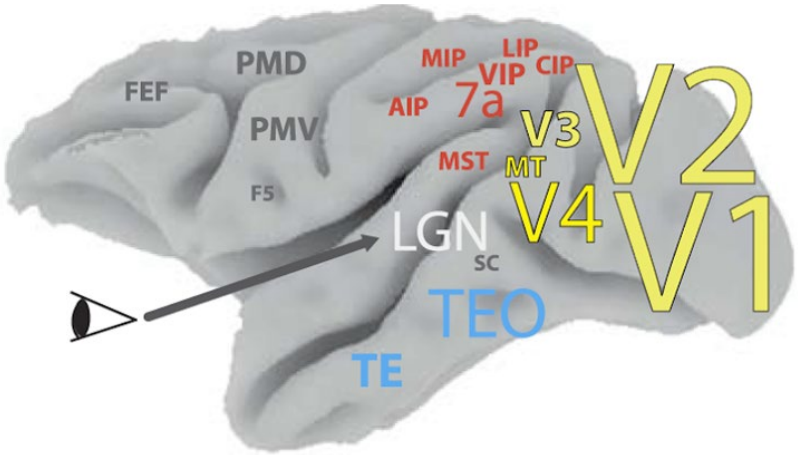
Motivación



Introducción a redes Neuronales Convolucionales

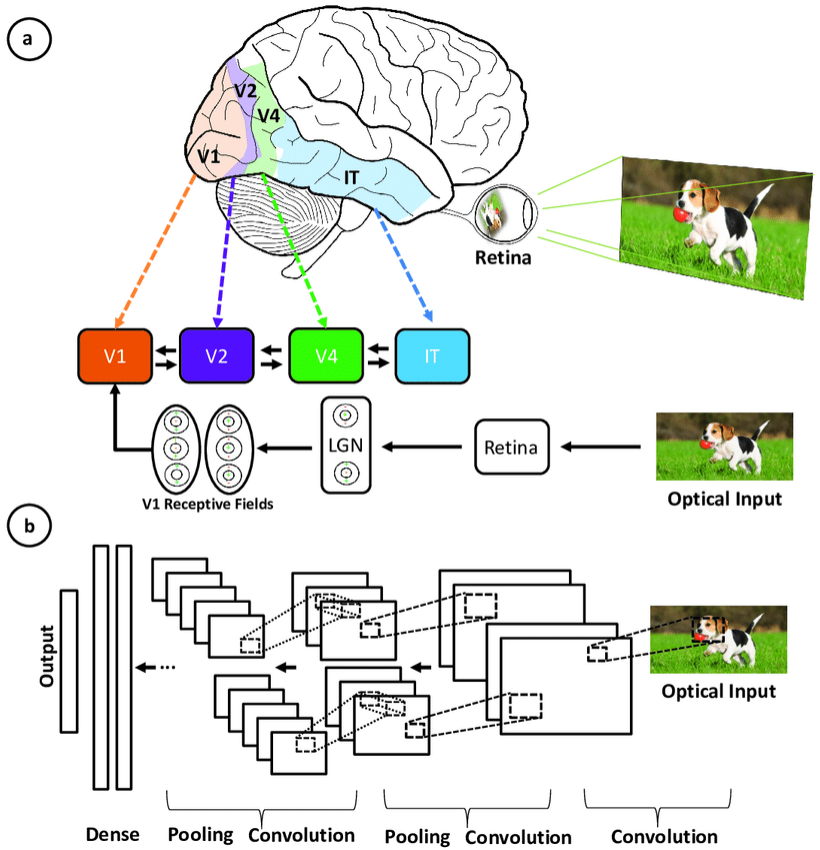
Corteza Visual





Introducción a redes Neuronales Convolucionales

Corteza Visual



EEG + Imágenes

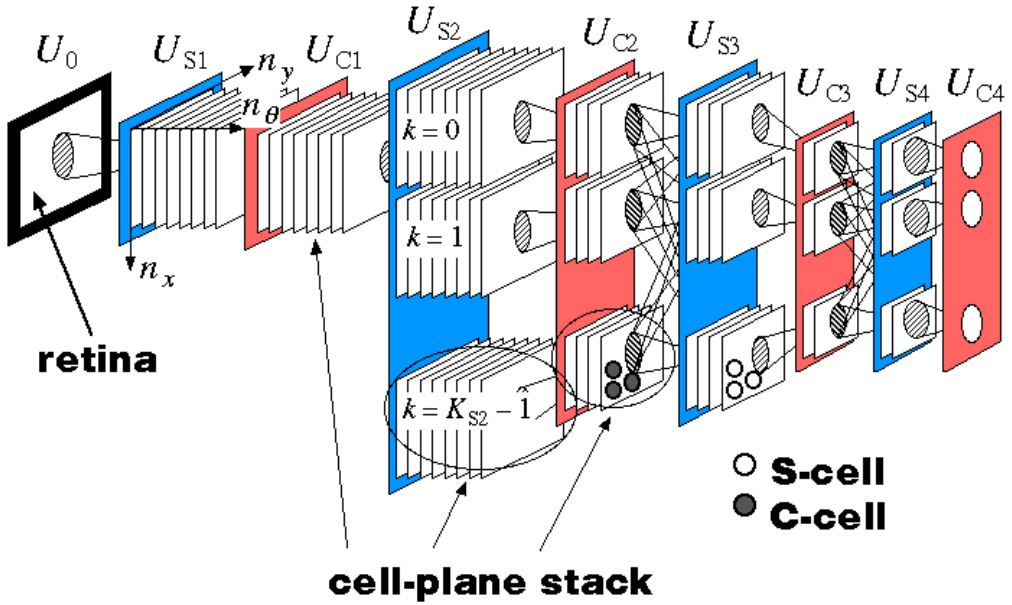
Tomada de: https://www.researchgate.net/figure/llustration-of-the-corrispondence-between-the-areas-associated-with-the-primary-visual_fig7_317679065

Introducción a redes Neuronales Convolucionales

Neocognitron - Fukushima (1980)

Neocognitron

Un modelo de red neuronal autoorganizada para un mecanismo de reconocimiento de patrones el cual no se ve afectado por el cambio de posición.

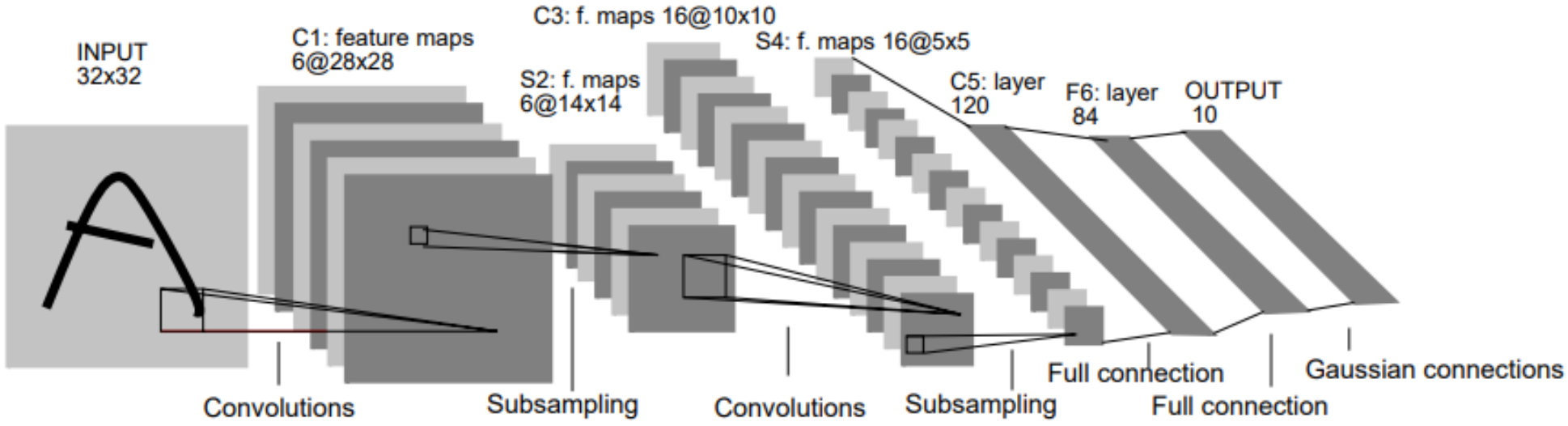


Fukushima, K. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. Biol. Cybernetics 36, 193–202 (1980).

Introducción a redes Neuronales Convolucionales

LeNet - LeCun (1998)

LeNet-5 → Arquitectura de la Red Neuronal Convolucional **LeNet-5** de LeCun (1998) para el reconocimiento de dígitos.

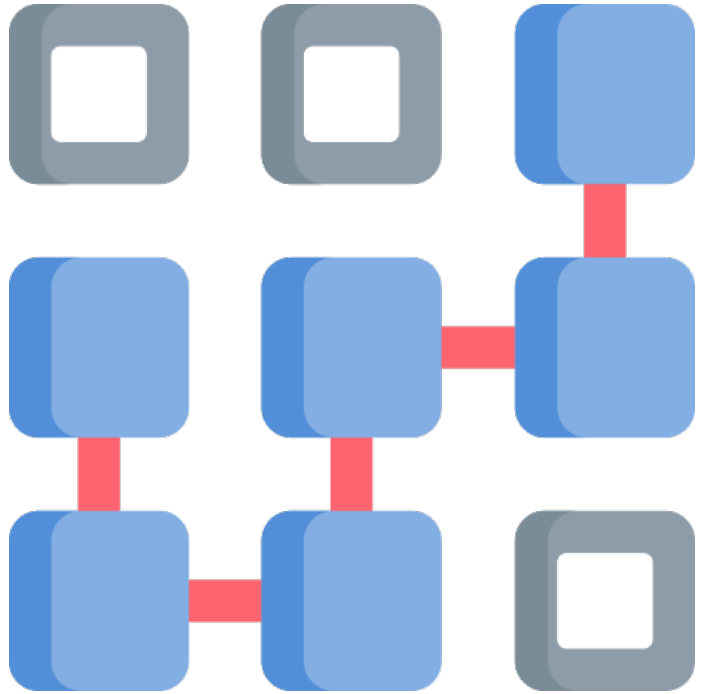


LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE (p./pp. 2278--2324), .

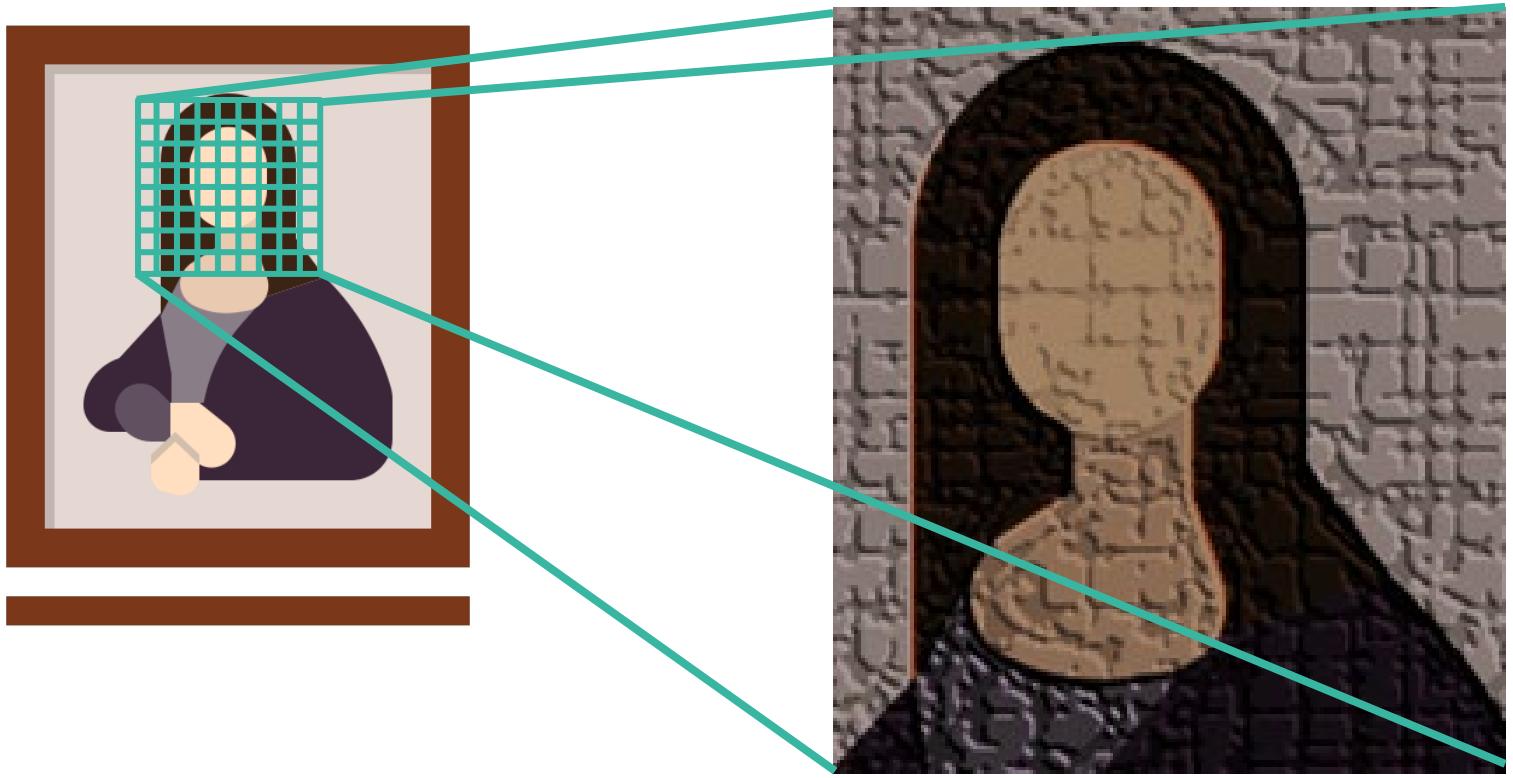
Demo LeNet

Agenda

- Introducción a Redes Neuronales Convolucionales.
- Capas Convolucionales y de Pooling.
- Data Augmentation.
- Transfer Learning
- Fine Tuning.
- Transfer Learning y Fine Tuning en Keras



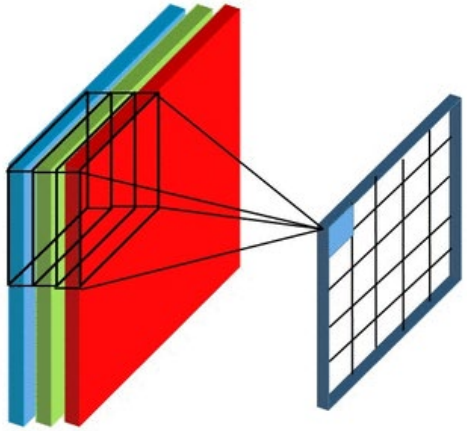
2 Capas Convolucionales y de Pooling



Capas Convolucionales y de Pooling

Capas importantes

Capas Convolucionales



Pooling

Feature Map

6	6	6	6
4	5	5	4
2	4	4	2
2	4	4	2

Max Pooling

Average Pooling

Sum Pooling

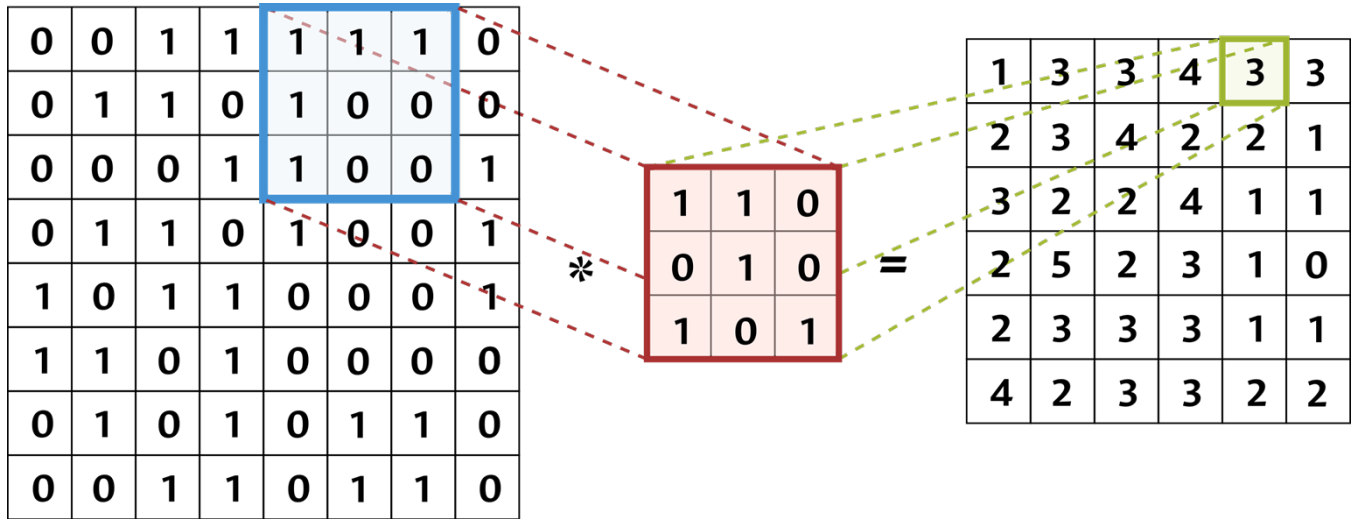
Para mejor visualización de las animaciones, dar Click en cada una de las imágenes.

Capas Convolucionales y de Pooling

Convolución

Capa Convolucional

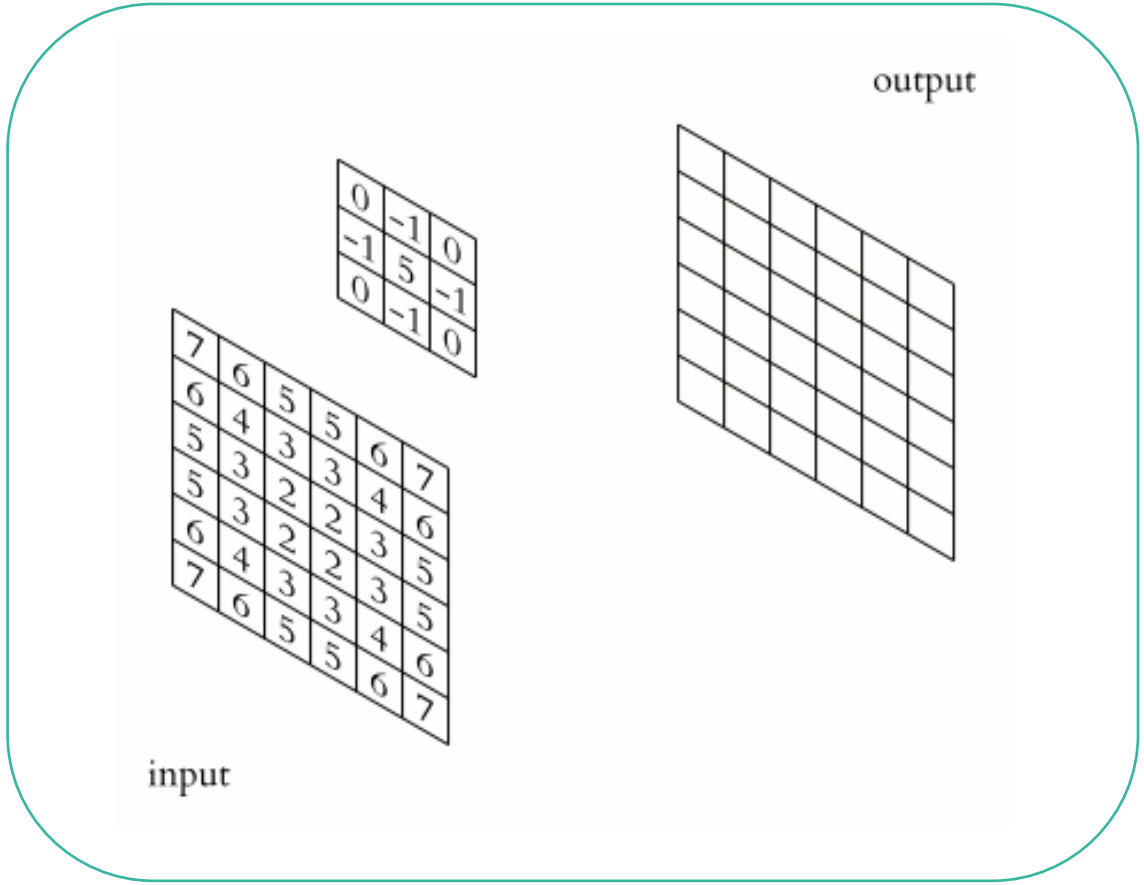
- Una Capa Convolucional es el principal bloque de construcción de una **CNN.**
- Contiene un conjunto de filtros (o kernels) cuyos parámetros deben aprenderse a lo largo del entrenamiento.



Capas Convolucionales y de Pooling

Convolución

Para mejor visualización de la animación, dar Click en la imagen.



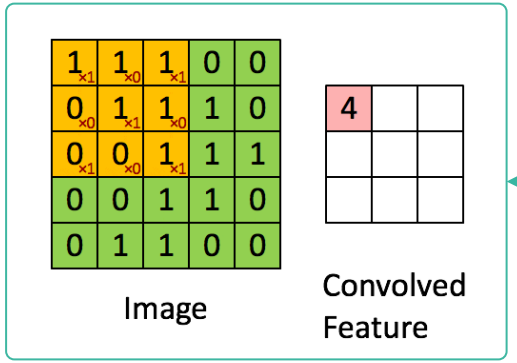
Cada filtro "**convoluciona**" con la imagen y crea un mapa de activación.

Capas Convolucionales y de Pooling

Convolución 2D

Parámetros

- **filters** : El número de filtros.
- **kernel_size** : Define el tamaño de los filtros.
- **strides**: Se trata del tamaño de salto de la convolución.
- **padding**: Define cómo se manejan los bordes de la imagen resultante.
  - **"valid"**: Resultado más pequeño.
  - **"same"**: Conserva el tamaño.



Para mejor visualización de la animación, dar Click en la imagen.

```
keras.layers.Conv2D(
                    filters=1,
                    kernel_size=(3,3),
                    strides=(1,1),
                    padding="valid",
)
```

Capas Convolucionales y de Pooling

Convolución 2D

Ejemplo

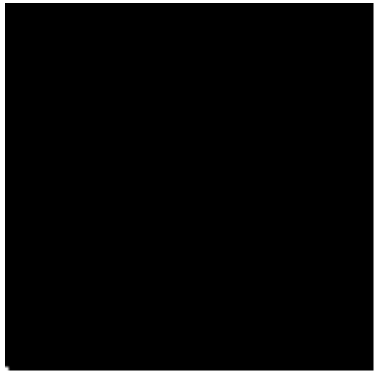


Filtro 1



Filtro 2





Filtro 3

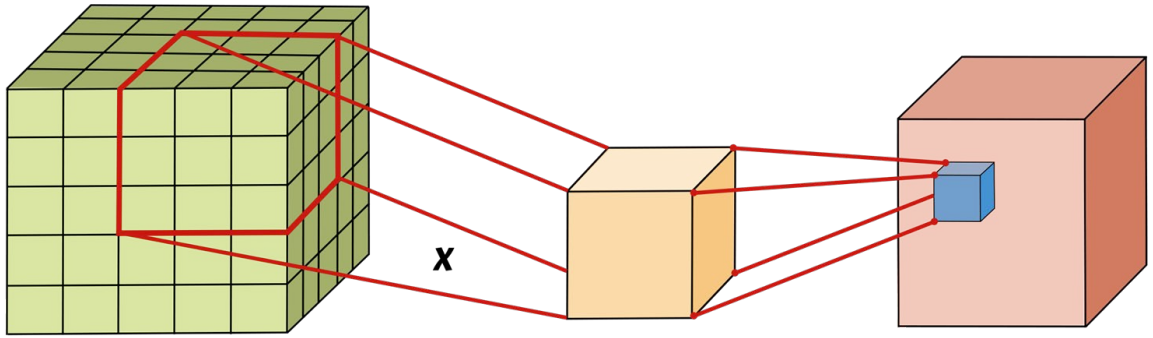


Filtro 4

Capas Convolucionales y de Pooling

Convolución 3D

Los parámetros de la convolución 3D son los mismos que en una convolución 2D !




```
keras.layers.Conv3D(
                    filters,
                    kernel_size,
                    strides,
                    padding,
)
```

- filters
- kernel_size
- strides
- padding

Capas Convolucionales y de Pooling

Pooling

Problema

La salida de las capas convolucionales registraran la posición precisa de las características en la entrada.

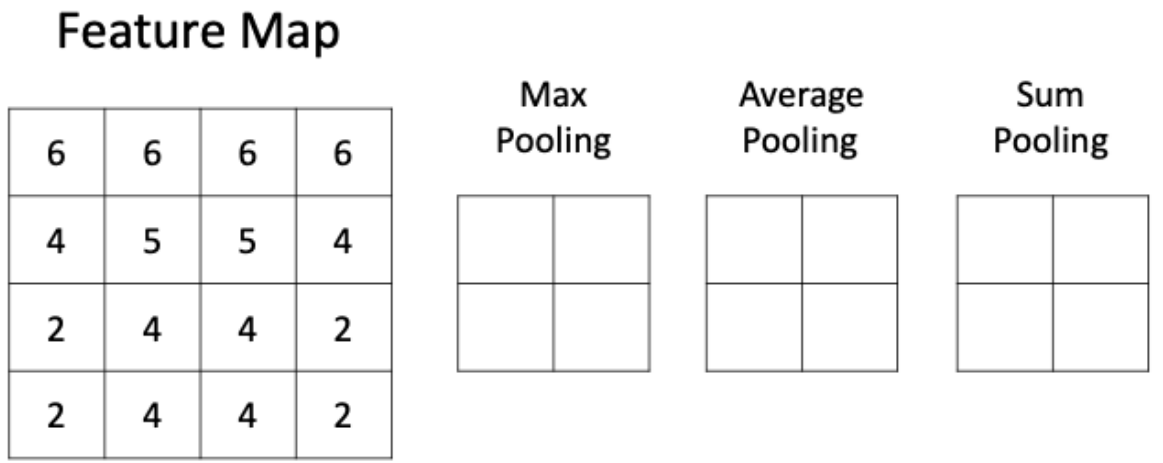
Es decir, pequeños movimientos en la posición de la característica en la imagen de entrada darán como resultado un mapa de características diferente

Solución

Down Sampling

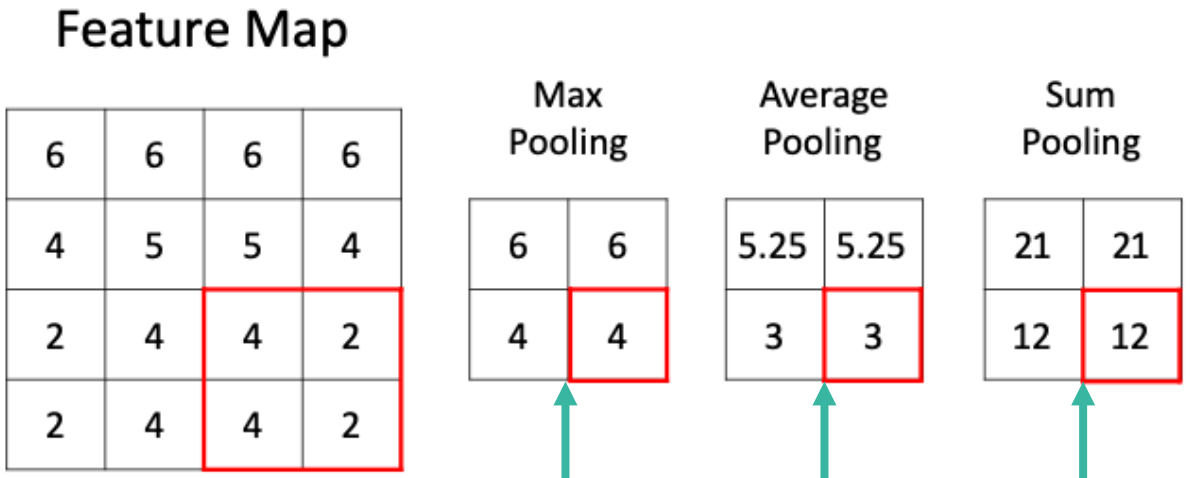
Se crea una versión de menor resolución de una señal de entrada que sigue conteniendo los elementos estructurales grandes o importantes, pero sin los detalles finos que pueden no ser tan útiles para realizar la tarea.

Para mejor visualización de la animación, dar Click en la imagen.



Capas Convolucionales y de Pooling

Pooling



Max Pooling

Toma el máximo valor en la ventana de muestreo.

Average Pooling

Toma el promedio de los valores en la ventana de muestreo

Sum Pooling

Toma la suma de los valores en la ventana de muestreo.

La capa de Pooling se aplica después de la convolución, especialmente después de la función de activación.

El Pooling actúa en cada canal o Feature Map por separado.

La salida conserva el número de Feature Maps.

Hay varias opciones:

Capas Convolucionales y de Pooling

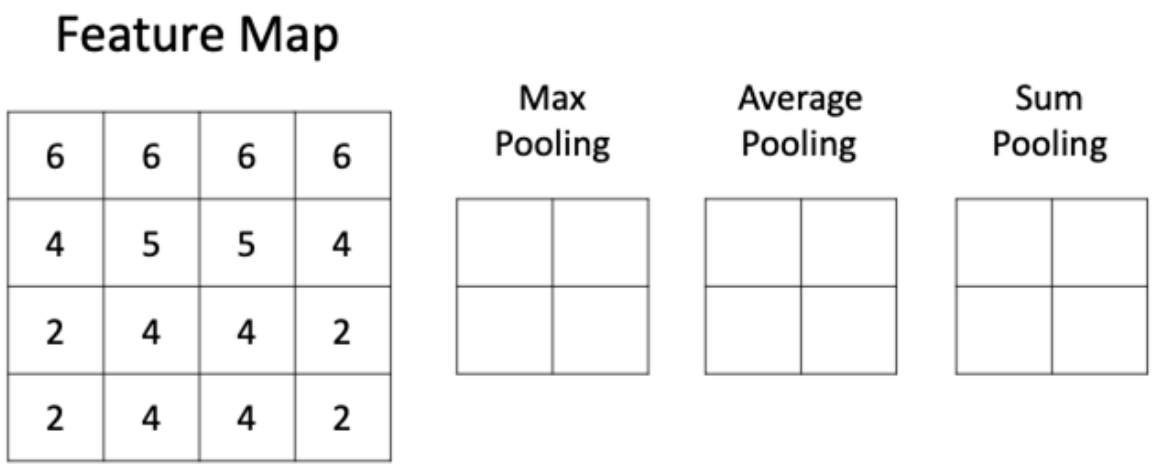
Pooling

```
keras.layers.MaxPooling2D(
                          pool_size=(2, 2),
                          strides=(2,2),
                          padding="valid",
)
```

```
keras.layers.AveragePooling2D(
                          pool_size=(2, 2),
                          strides=(2,2),
                          padding="valid",
)
```

- keras.layers.MaxPooling2D
- keras.layers.AveragePooling2D
- keras.layers.MaxPooling3D
- keras.layers.AveragePooling3D

Para mejor visualización de la animación, dar Click en la imagen.



Parametros

pool_size : Dimensiones de la ventana de muestreo.

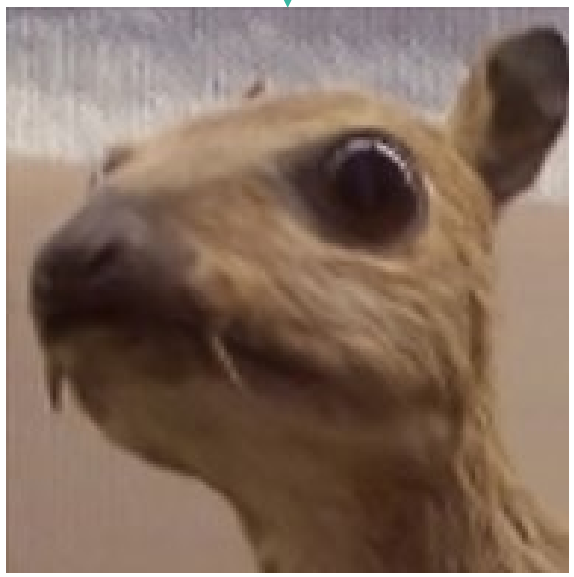
padding: Manejo de los bordes del *Feature Map*.

strides: Tamaño del paso de la ventana.

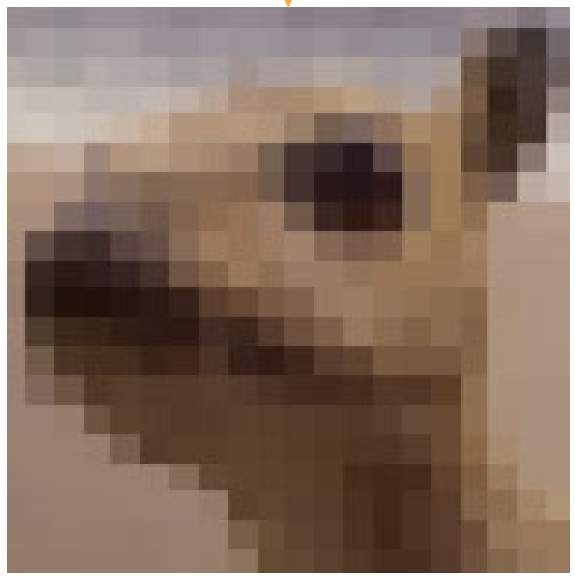
Capas Convolucionales y de Pooling

Pooling : Ejemplos

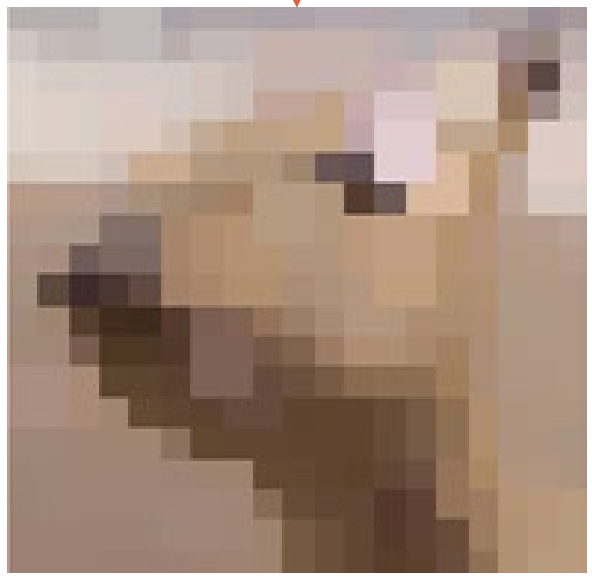
Imagen Original



Average Pooling



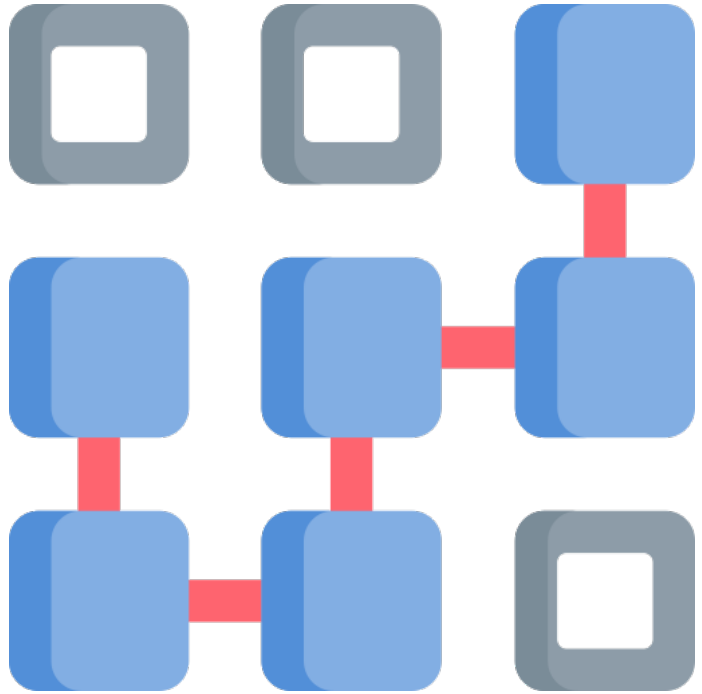
Maximum Pooling



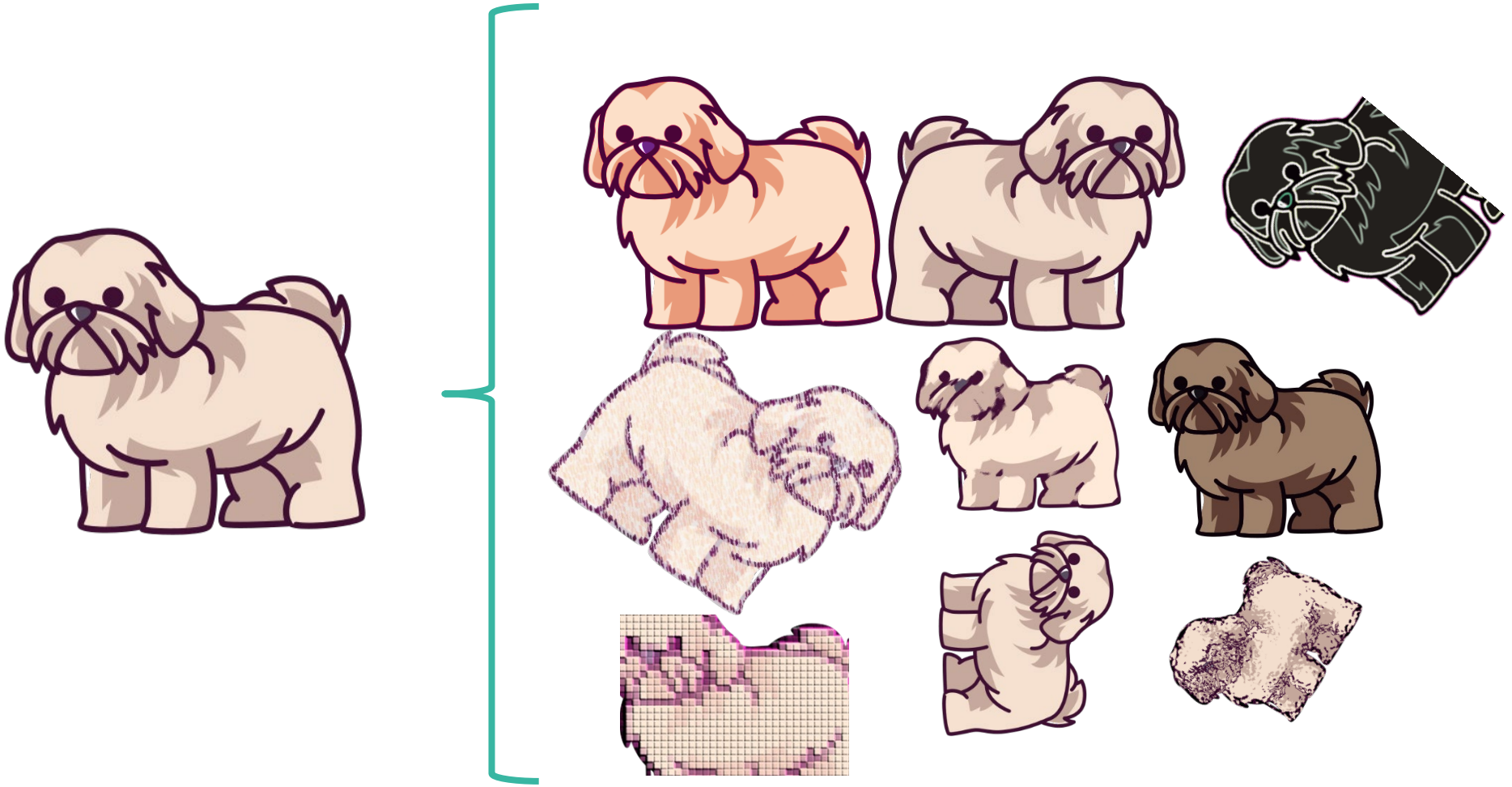
CNN Demo Video CNN

Agenda

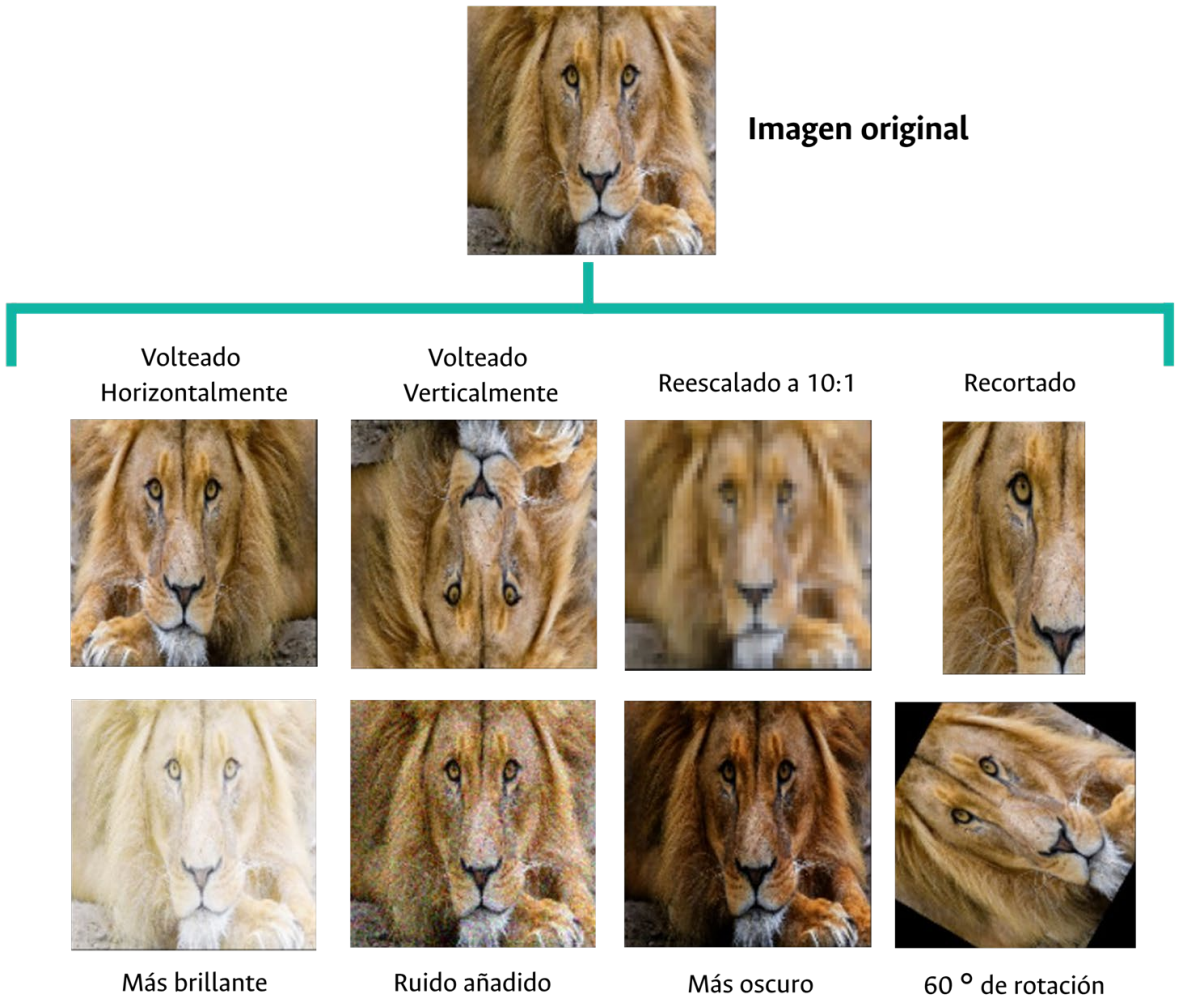
- Introducción a Redes Neuronales Convolucionales.
- Capas Convolucionales y de Pooling.
- Data Augmentation.
- Transfer Learning
- Fine Tuning.
- Transfer Learning y Fine Tuning en Keras



3 Data Augmentation



Data Augmentation



[Demo Data Augmentation]

Problema

Las redes neuronales convolucionales necesitan una gran cantidad de datos para aprender.

Solución

Extender el conjunto de entrenamiento por medio de la generación de datos o ***Data Augmentation***, en el caso específico de imágenes, consiste en generar nuevas imágenes transformadas aplicando: traslaciones, rotaciones, intensidad, contraste y muchos más.

Data Augmentation

Ventajas

- Ayuda a prevenir *overfitting*.
- Permite entrenar modelos cuando hay pocos datos.
- Ayuda a mejorar la robustez del modelo.
- Puede ayudar a mejorar el desempeño del modelo.

Importante !

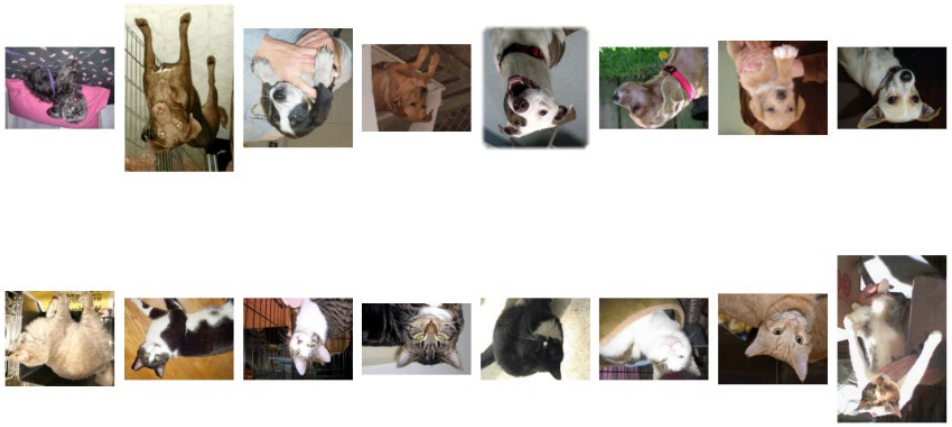
- Si hay sesgos en el dataset original, persistirán en el dataset aumentado.
- Encontrar los parámetros adecuados de un buen *Augmentation* puede ser retador.
- *Data Augmentation* solo se aplica en el conjunto de entrenamiento, no en validación ni en prueba.

Data Augmentation

Data Augmentation

Keras

Keras provee herramientas que nos permiten realizar *Data Augmentation* y que pueden ser utilizadas en conjunto con los modelos.





Data Augmentation

Capas de Image Augmentation de Keras

Transformaciones geométricas

- RandomFlip
- RandomRotation
- RandomZoom
- RandomTranslation
- RandomShear
- RandomCrop
- RandomPerspective

Ajustes de color y brillo

- RandomBrightness
- RandomContrast
- RandomHue
- RandomSaturation
- RandomColorJitter
- RandomGrayscale
- RandomInvert
- Solarization
- Equalization

Desenfoque y nitidez

- RandomGaussianBlur
- RandomSharpness

Capas disponibles

Data Augmentation

Image Data Generator

```
train_aug = keras.Sequential(
                                [
                                    keras.layers.Input(shape=(None, None, 3)),
                                    keras.layers.Rescaling(scale=1./255),
                                    keras.layers.RandomFlip("horizontal"),
                                    keras.layers.RandomRotation(factor=0.02),
                                    keras.layers.RandomZoom(height_factor=0.2, width_factor=0.2),
                                ], name="train_aug")
```



Imagen original



Data Augmentation

Image Data Generator : Código

```python
train_aug = keras.Sequential(
                            [
                                keras.layers.Input(shape=(None, None, 3)),
                                keras.layers.Rescaling(scale=1./255),
                                keras.layers.RandomFlip("horizontal"),
                                keras.layers.RandomRotation(factor=0.02),
                                keras.layers.RandomZoom(height_factor=0.2, width_factor=0.2),
                            ], name="train_aug")
```

Esto define un generador de imágenes. Aquí se definen los parámetros del *Data Augmentation*.

```python
train_generator = keras.utils.image_dataset_from_directory(directory=train_dir,
                                                           image_size=(150, 150),
                                                           batch_size=batch_size,
                                                           labels ='inferred')
```

Aquí se definen varias cosas :

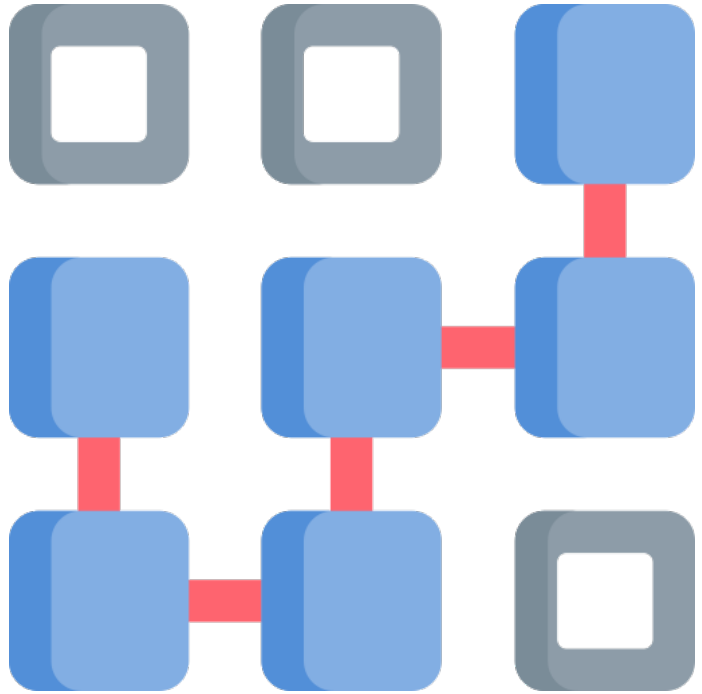
- El directorio donde se encuentran las imágenes.
- El tamaño final de las imágenes.
- El tamaño del batch.
- Cómo manejar las etiquetas.

```python
model.fit( train_generator,
           steps_per_epoch=...,
           epochs=...
         )
```

El generador se pasa como argumento al método *fit* del modelo.

Agenda

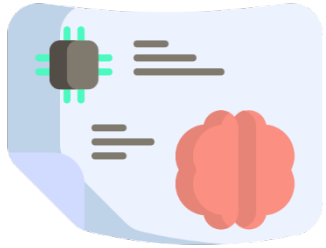
- Introducción a Redes Neuronales Convolucionales.
- Capas Convolucionales y de Pooling.
- Data Augmentation.
- Transfer Learning
- Fine Tuning.
- Transfer Learning y Fine Tuning en Keras



4 Transfer Learning

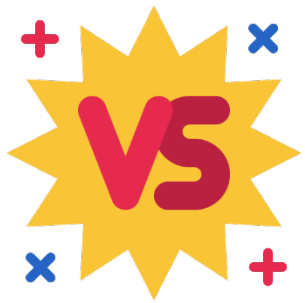


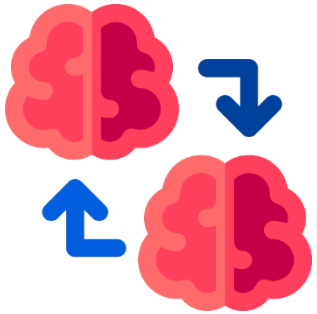
Motivación



Machine Learning Tradicional

- Aprendizaje aislado, de una sola tarea.
- El conocimiento no se retiene ni se acumula.
- El aprendizaje se realiza teniendo en cuenta los conocimientos adquiridos en otras tareas.



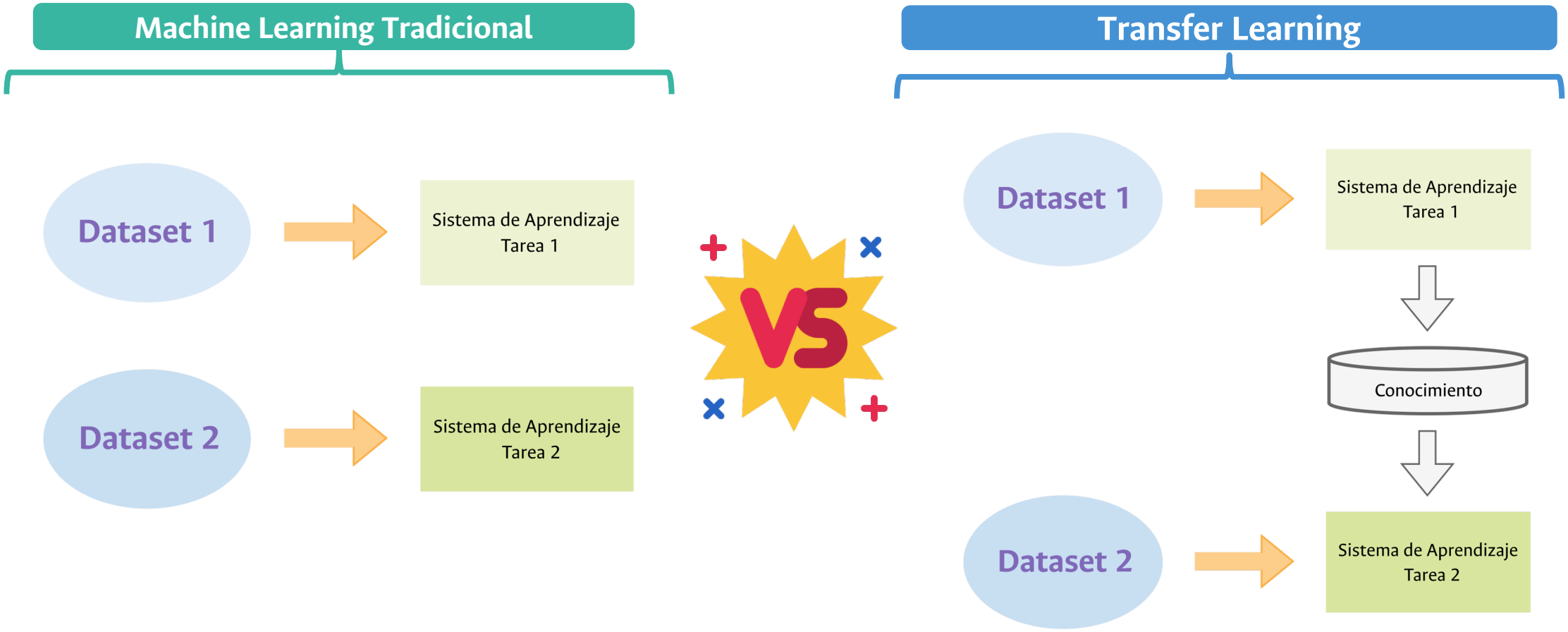


Transfer Learning

- El aprendizaje de una nueva tarea se basa en las tareas aprendidas anteriormente
- El proceso de aprendizaje puede ser más rápido, más preciso y/o necesitar menos datos de entrenamiento.

Transfer Learning

Motivación



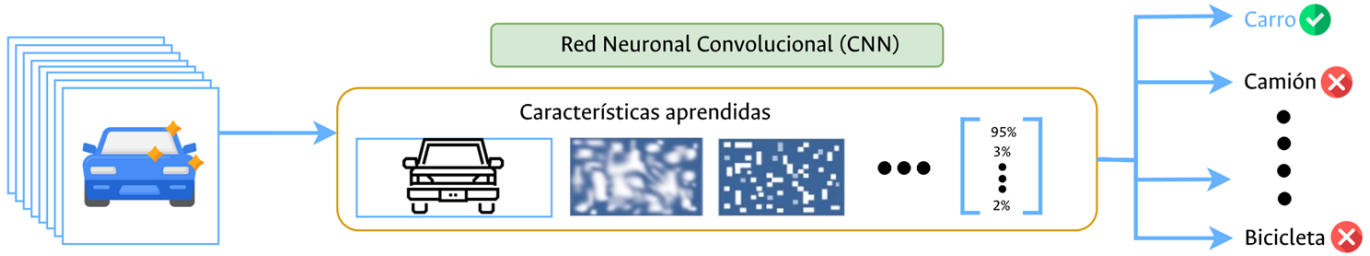
Transfer Learning

Motivación

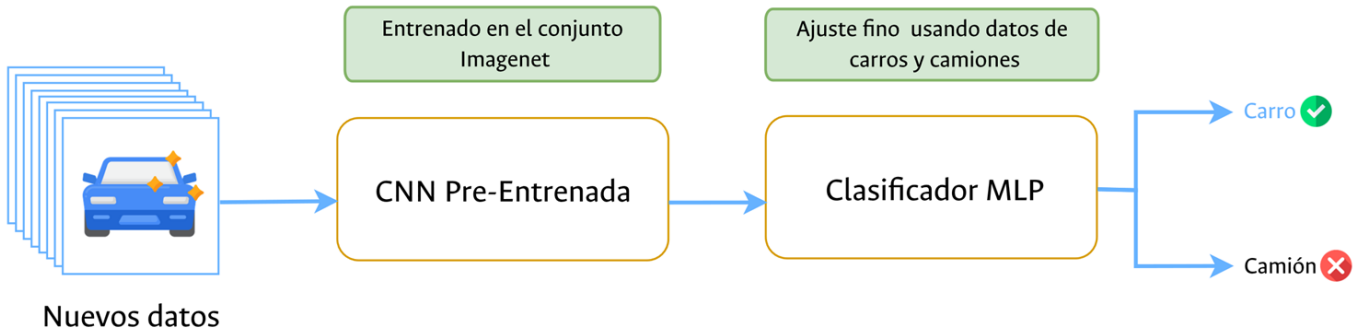
Transfer learning se usa

- Para ahorrar tiempo y recursos al no tener que entrenar múltiples modelos desde cero para completar tareas similares.
- Por eficiencia al requerir grandes cantidades de recursos, como la categorización de imágenes o el procesamiento del lenguaje natural.
- Para compensar la falta de datos de entrenamiento etiquetados en una organización, mediante el uso de modelos pre-entrenados.

Entrenamiento desde cero



Transfer Learning

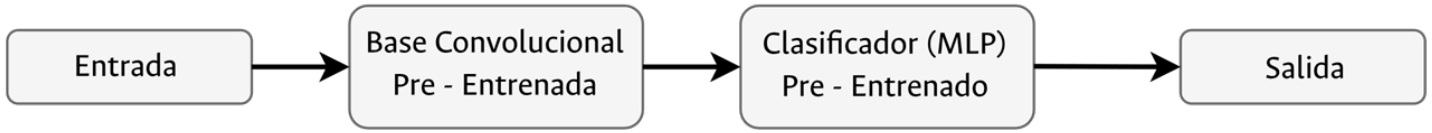


Transfer Learning

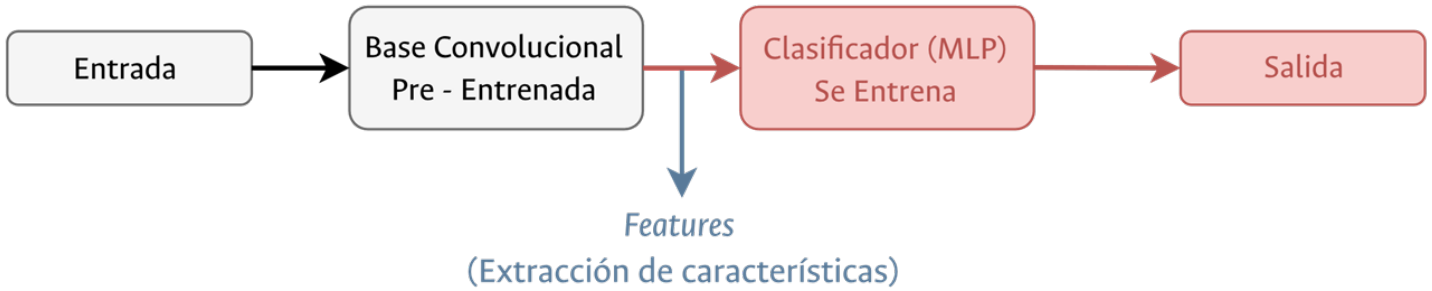
¿Cómo funciona?

Funciona transfiriendo tantos conocimientos como sea posible de un modelo existente a un nuevo modelo diseñado para una tarea similar.

Modelo Pre - Entrenado



Transfer Learning (Extracción de características)



Se escoge un modelo pre-entrenado en una tarea similar.

Se conserva la parte del modelo encargado de aprender la representación en un espacio latente: **el extractor de características.**

La parte final, por lo general es un MLP, el cual se entrena.

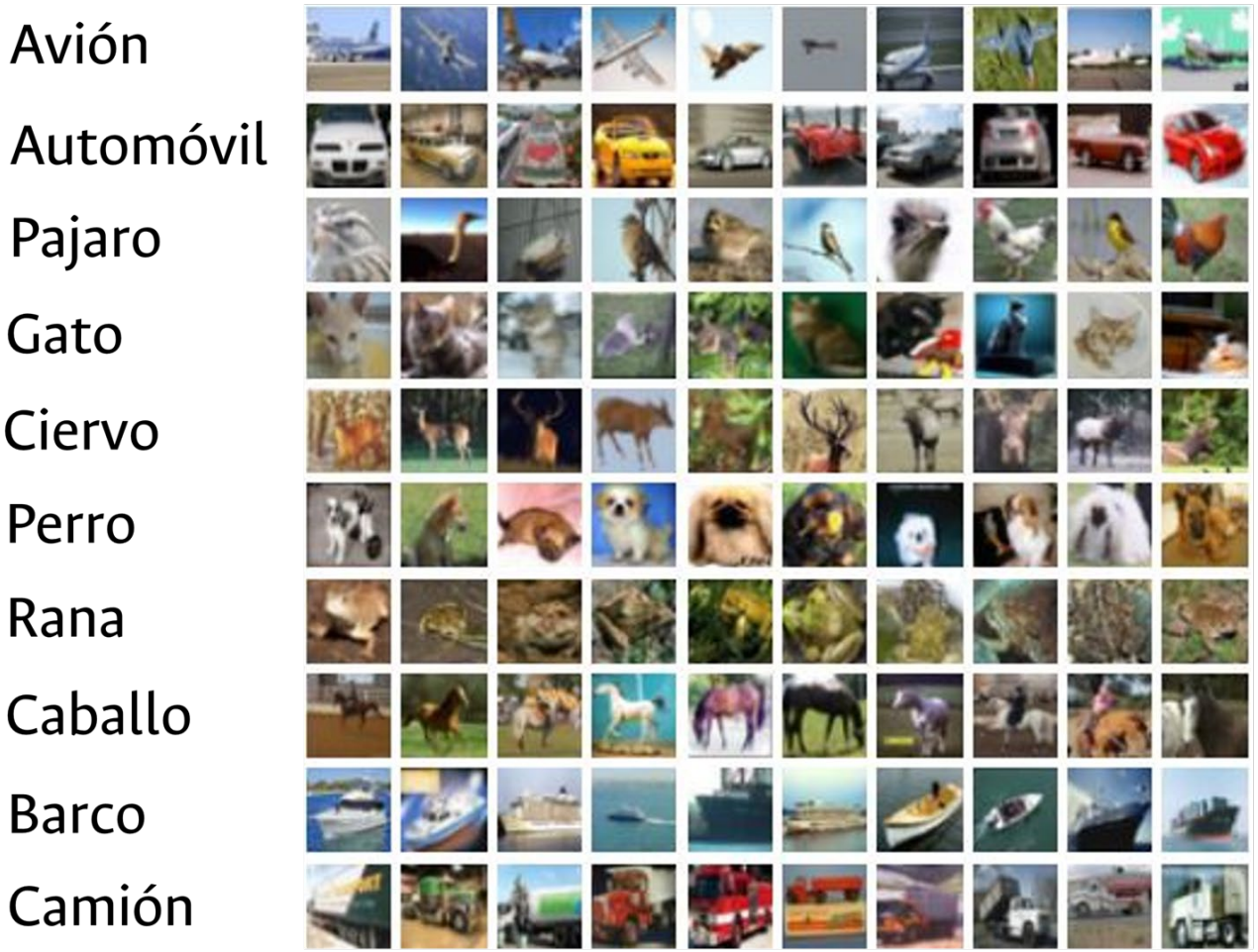
Transfer Learning

Modelos Pre – entrenados en Keras

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0

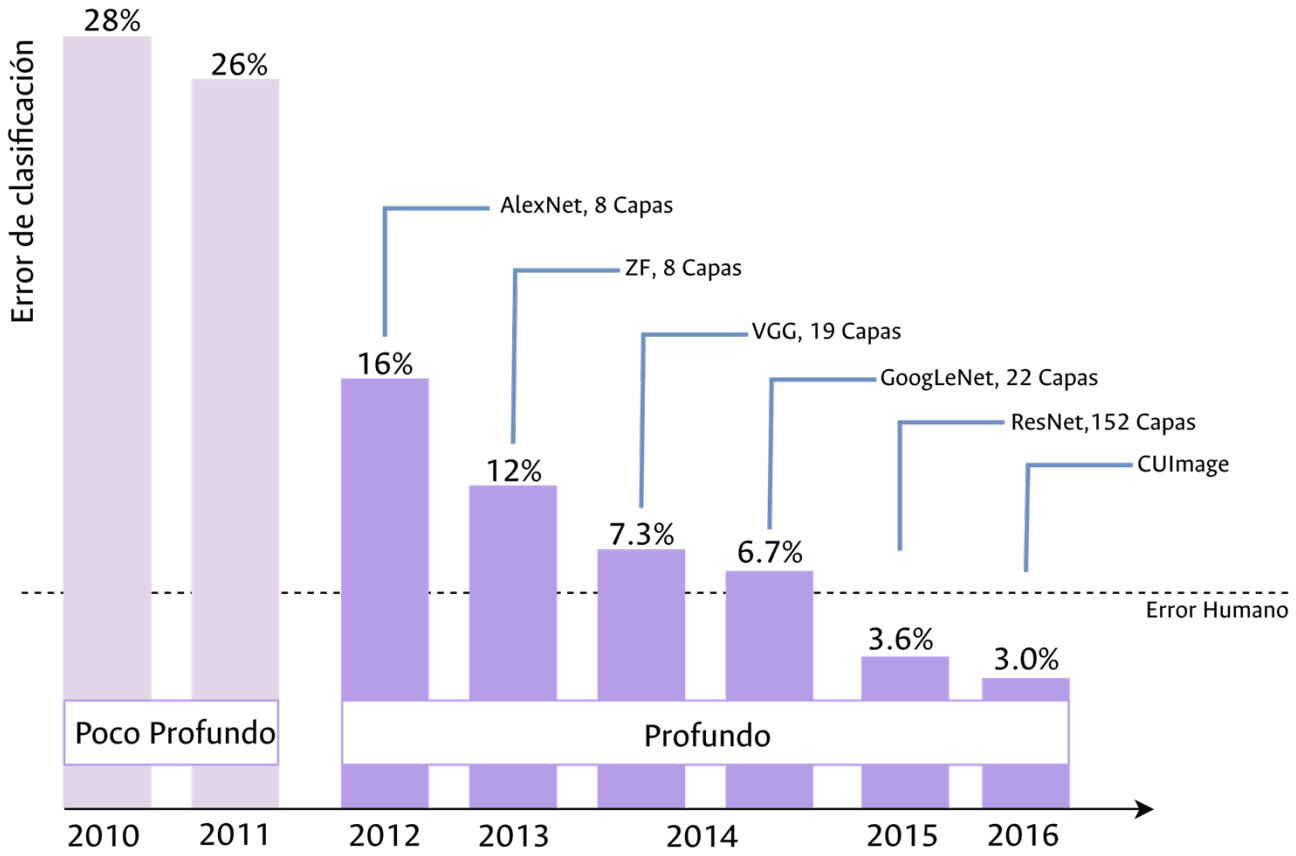
Transfer Learning

Modelos Pre – entrenados en Keras



Transfer Learning

Modelos Pre – entrenados en Keras



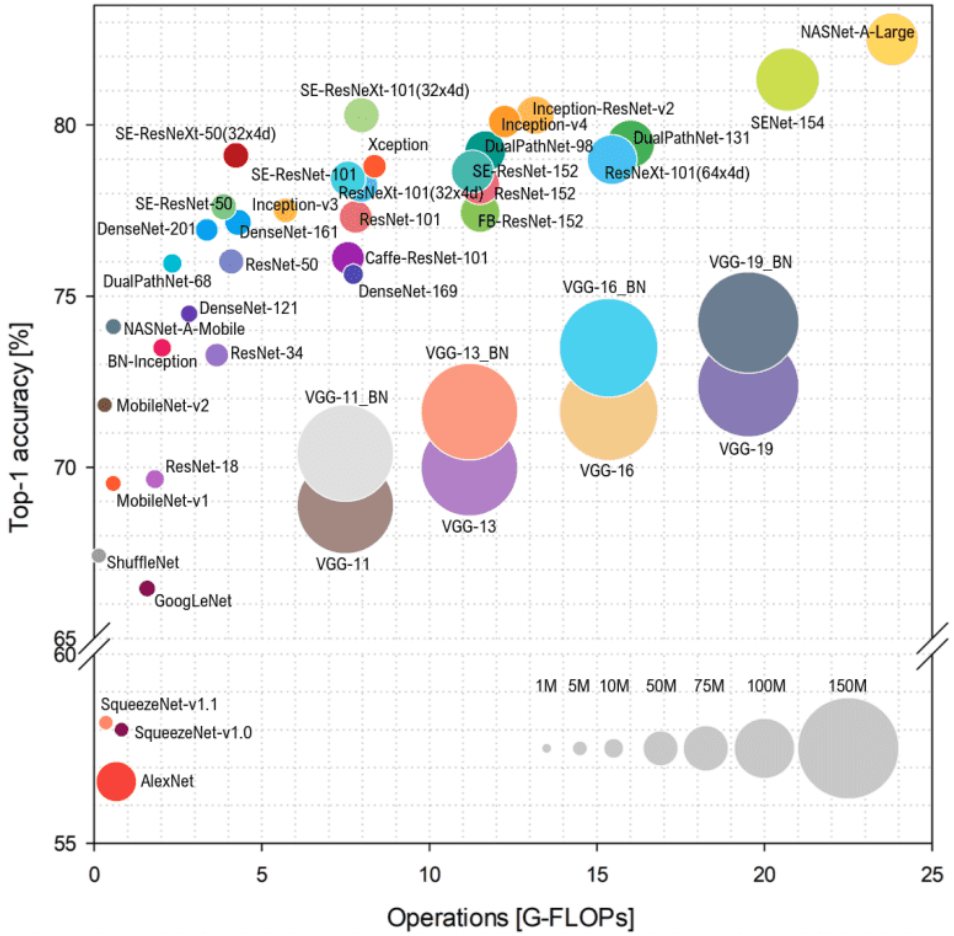
ImageNet tiene una competencia anúal : **The ImageNet Large Scale Visual Recognition Challenge (ILSVRC).**

- Distintos métodos compiten para **clasificar y detectar objetos** de forma correcta.
- A partir del **año 2012**, los métodos de Deep Learning comenzaron a ganar atención cuando una arquitectura de red convolucional conocida como **AlexNet** ganó la competencia.
- En el **año 2015** una arquitectura de red neuronal (**ResNet**) demostró tener un error menor en comparación con un ser humano., mediante el uso de modelos pre-entrenados.

Transfer Learning

Modelos Pre – entrenados en Keras

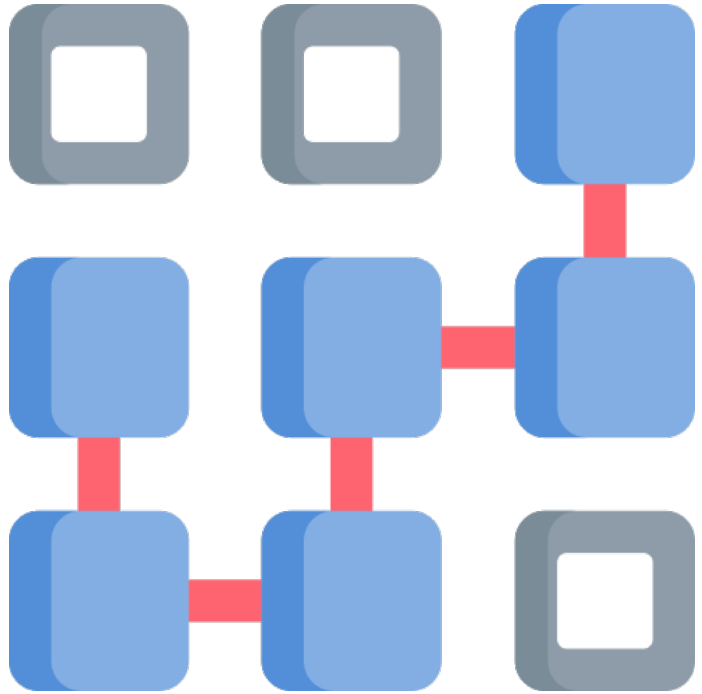
Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0



[Demo Mediapipe - Google]

Agenda

- Introducción a Redes Neuronales Convolucionales.
- Capas Convolucionales y de Pooling.
- Data Augmentation.
- Transfer Learning
- Fine Tuning.
- Transfer Learning y Fine Tuning en Keras



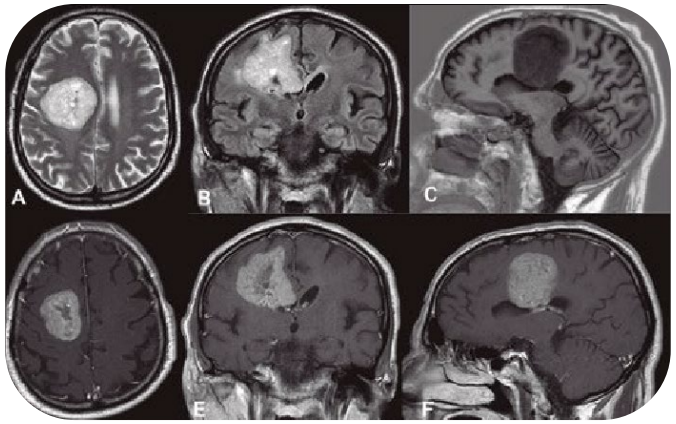
5 Fine Tuning



Fine Tuning

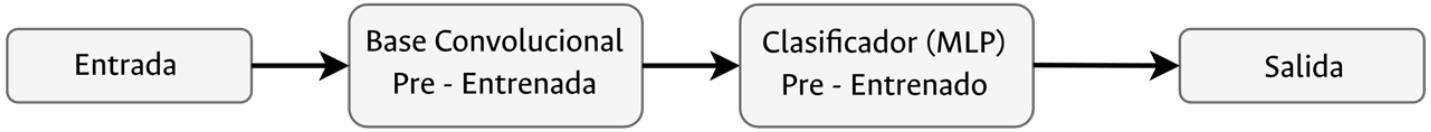
Algunas veces la tarea requiere un poco más de esfuerzo.

Por ejemplo, cuando tenemos un conjunto de datos muy diferente a ImageNet:

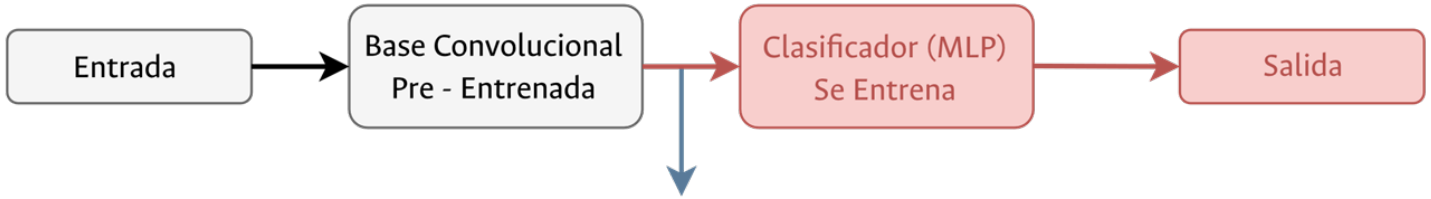


Fine Tuning

Modelo Pre - Entrenado

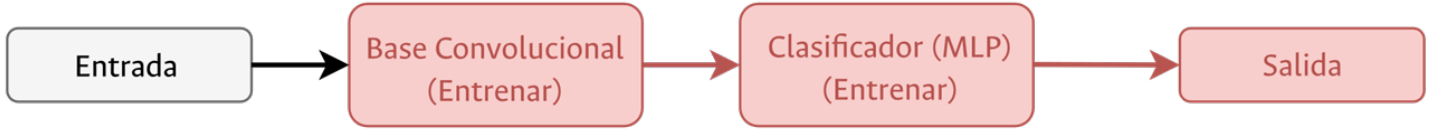


Transfer Learning (Extracción de características)



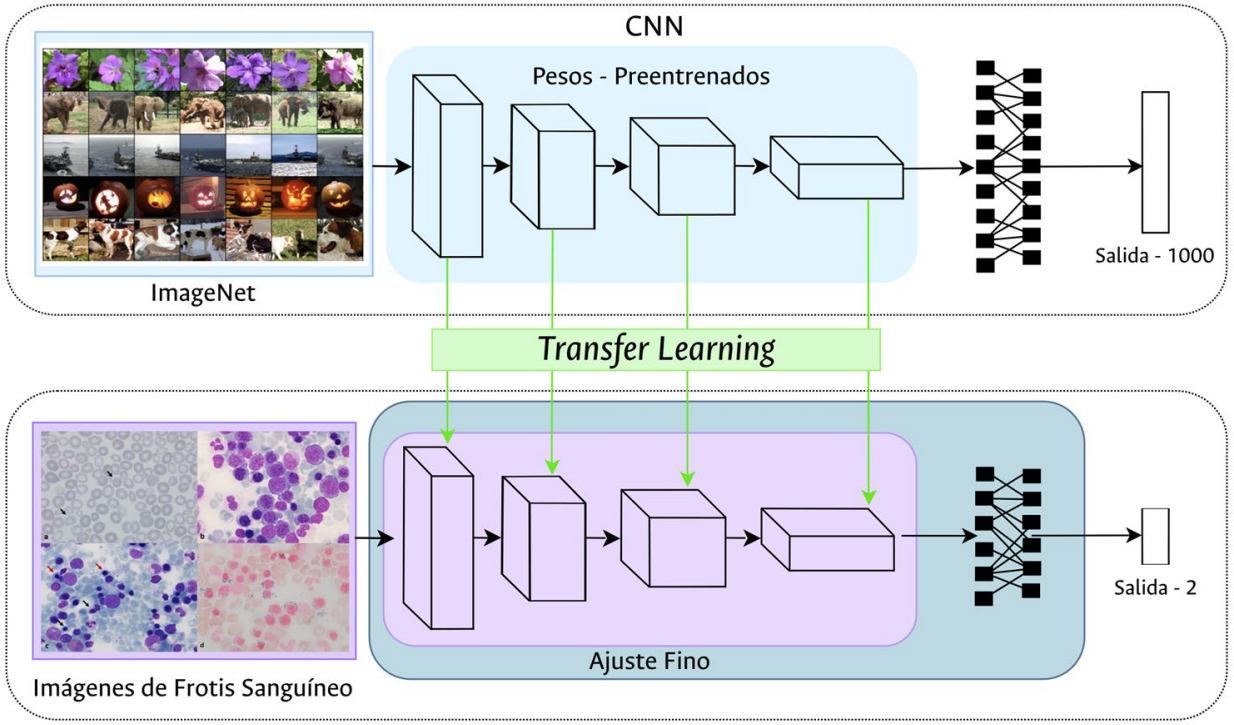
Features
(Extracción de características)

Fine Tuning o Ajuste Fino



Fine Tuning

¿Cómo funciona?



Vogado, L., Veras, R., Aires, K., Araújo, F., Silva, R., Ponti, M., & Tavares, J. M. R. S. (2021). Diagnosis of Leukaemia in Blood Slides Based on a Fine-Tuned and Highly Generalisable Deep Learning Model. Sensors (Basel, Switzerland), 21(9), 2989. https://doi.org/10.3390/s21092989

TeachableMachine - Google

Se escoge un modelo pre-entrenado.

Warming Up (calentamiento):

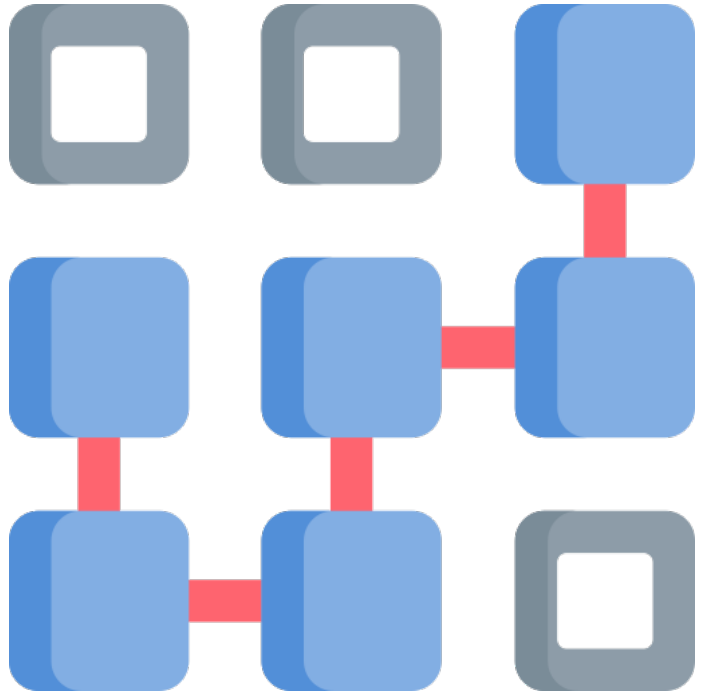
- Se congela el bloque convolucional.
- La parte final (que por lo general es un MLP) se entrena durante pocas epochs.

Luego, se liberan los pesos de todo el modelo, y se entrena, con un learning rate muy pequeño :

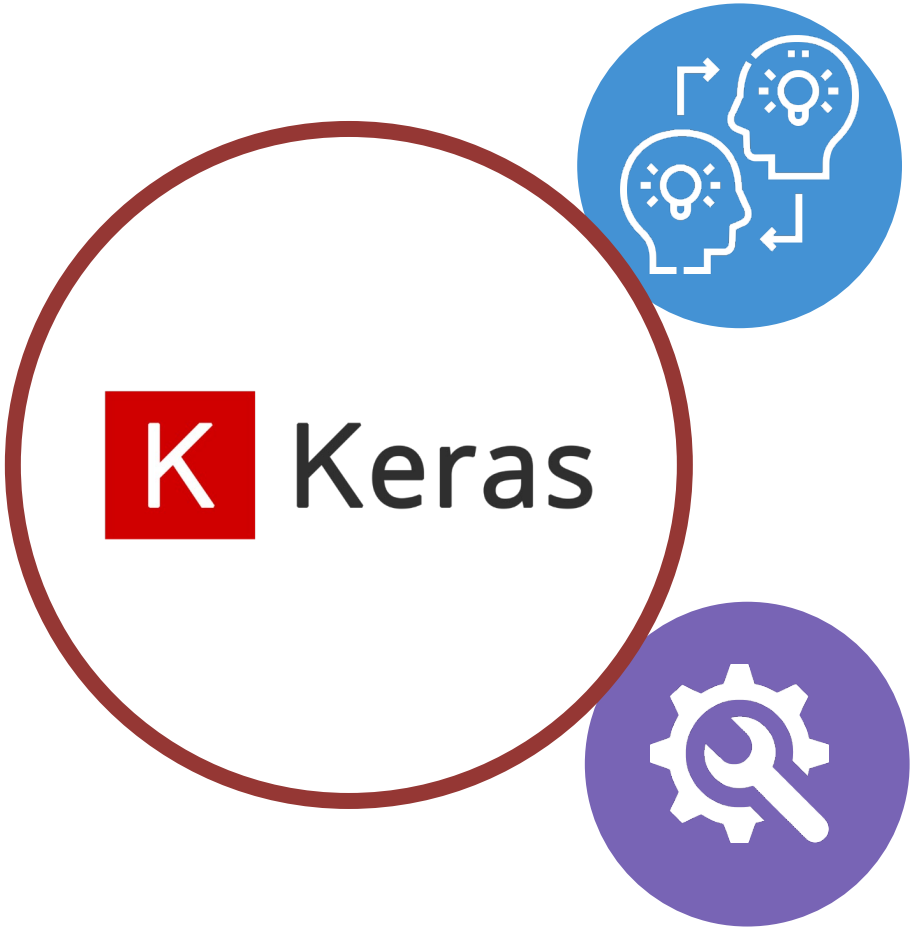
- Queremos lograr un ajuste fino, muy sutil.
- **Un learning rate grande puede dañar todos los pesos.**

Agenda

- Introducción a Redes Neuronales Convolucionales.
- Capas Convolucionales y de Pooling.
- Data Augmentation.
- Transfer Learning
- Fine Tuning.
- Transfer Learning y Fine Tuning en Keras



6 Transfer Learning y Fine Tuning en Keras



Transfer Learning y Fine Tuning en Keras

Definir un modelo Pre - entrenado

```
model =
keras.applications.MobileNet(weights='imagenet',
                              include_top=True,
                              input_shape=(224, 224, 3))
```

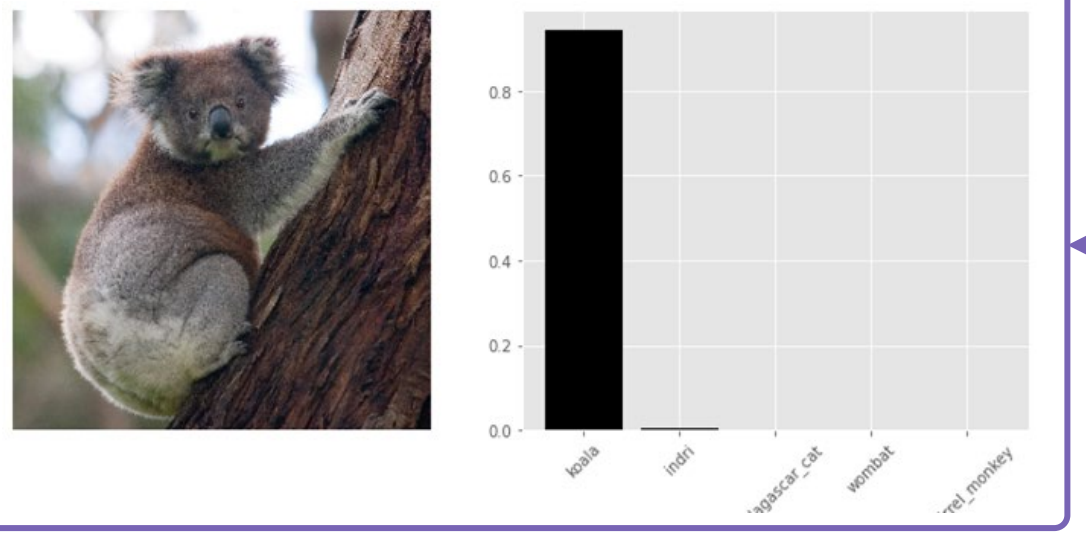
Se cargan el modelo con los pesos del entrenamiento con ImageNet.

Se incluye en el modelo el clasificador final de 1000 clases de ImageNet.

Se define el tamaño de las imágenes de entrada.

Podemos usar esto para hacer directamente predicciones.

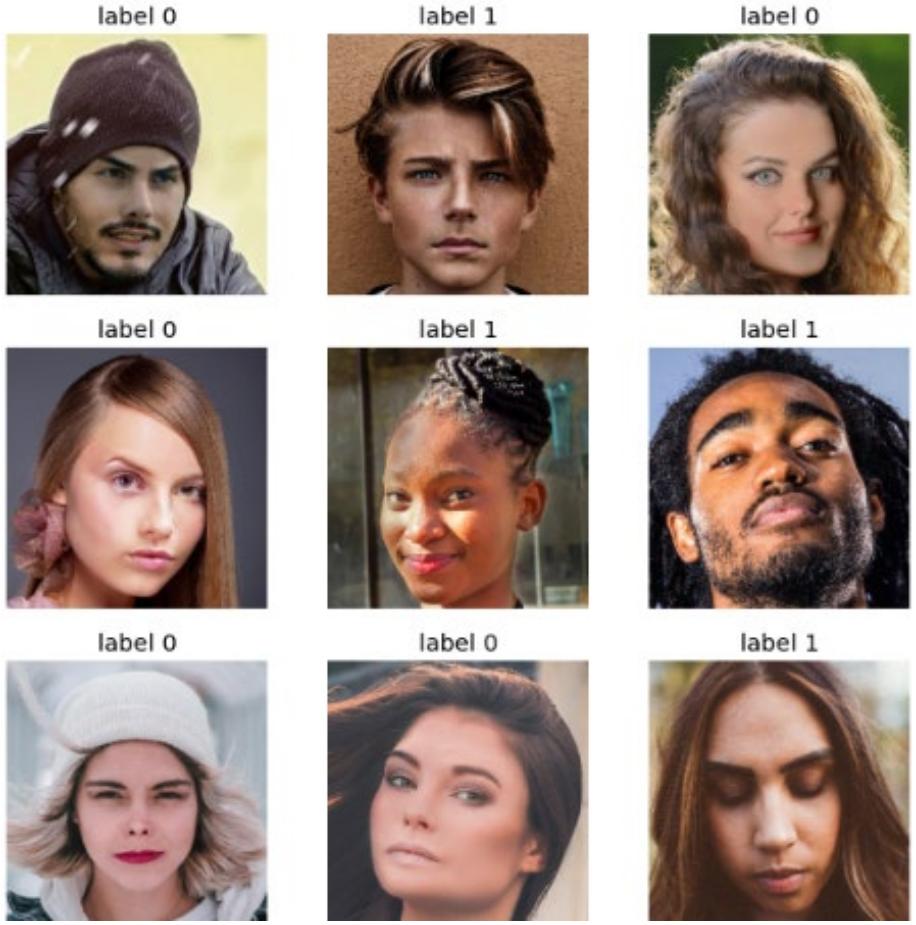
```
model.predict(...)
```



Transfer Learning y Fine Tuning en Keras

Definir un modelo Pre - entrenado

Para ejemplificar el *Transfer Learning* se realizará un problema de clasificación de dos clases



Transfer Learning y Fine Tuning en Keras

Definir un modelo Pre - entrenado

Transfer Learning

```
extractor =
keras.applications.MobileNet(weights='imagenet',
                             include_top=False,
                             input_shape=(224, 224, 3))
```

- Se cargan el modelo con los pesos del entrenamiento con ImageNet.
- **No se incluye** en el modelo el clasificador final de 1000 clases.
- Se define el tamaño de las imágenes de entrada.

```
for layer in extractor.layers:
    layer.trainable=False
```

- El extractor no se va a entrenar; se congela.

```
pool = keras.layers.GlobalAveragePooling2D()(extractor.output)
dense1 = keras.layers.Dense(units=32, activation="relu")(pool)
drop1= keras.layers.Dropout(0.2)(dense1)
dense2 = keras.layers.Dense(units=1, activation="sigmoid")(drop1)

tl_model = keras.models.Model(inputs=[extractor.input], outputs=[dense2])
```

- Agregamos capas densas nuevas.
- Es un MLP.
- La entrada son las features dadas por la CNN.
- La salida es de dos clases.
- Estas capas no están entrenadas.

Transfer Learning y Fine Tuning en Keras

Definir un modelo Pre - entrenado

Compilación

```
tl_model.compile(loss="binary_crossentropy",
    optimizer=keras.optimizers.Adam(lr=1e-3),
    metrics=["accuracy"])
```

El modelo se compila como siempre.

Transfer Learning y Fine Tuning en Keras

Definir un modelo Pre - entrenado

block7a_project_bn (BatchNormalizatio…	(None, 7, 7, 320)	1,280	block7a_project_…
top_conv (Conv2D)	(None, 7, 7, 1280)	409,600	block7a_project_…
top_bn (BatchNormalizatio…	(None, 7, 7, 1280)	5,120	top_conv[0][0]
top_activation (Activation)	(None, 7, 7, 1280)	0	top_bn[0][0]
global_average_poo… (GlobalAveragePool…	(None, 1280)	0	top_activation[0…
dense (Dense)	(None, 32)	40,992	global_average_p…
dropout (Dropout)	(None, 32)	0	dense[0][0]
dense_1 (Dense)	(None, 1)	33	dropout[0][0]

Total params: 4,090,596 (15.60 MB)
Trainable params: 41,025 (160.25 KB)
Non-trainable params: 4,049,571 (15.45 MB)

Este es el extractor de características del modelo, cuyos pesos no se entrenan

Este es el MLP nuevo, el cual si se entrenará.

No son muchos los parámetros para entrenar.

Visualize CNN

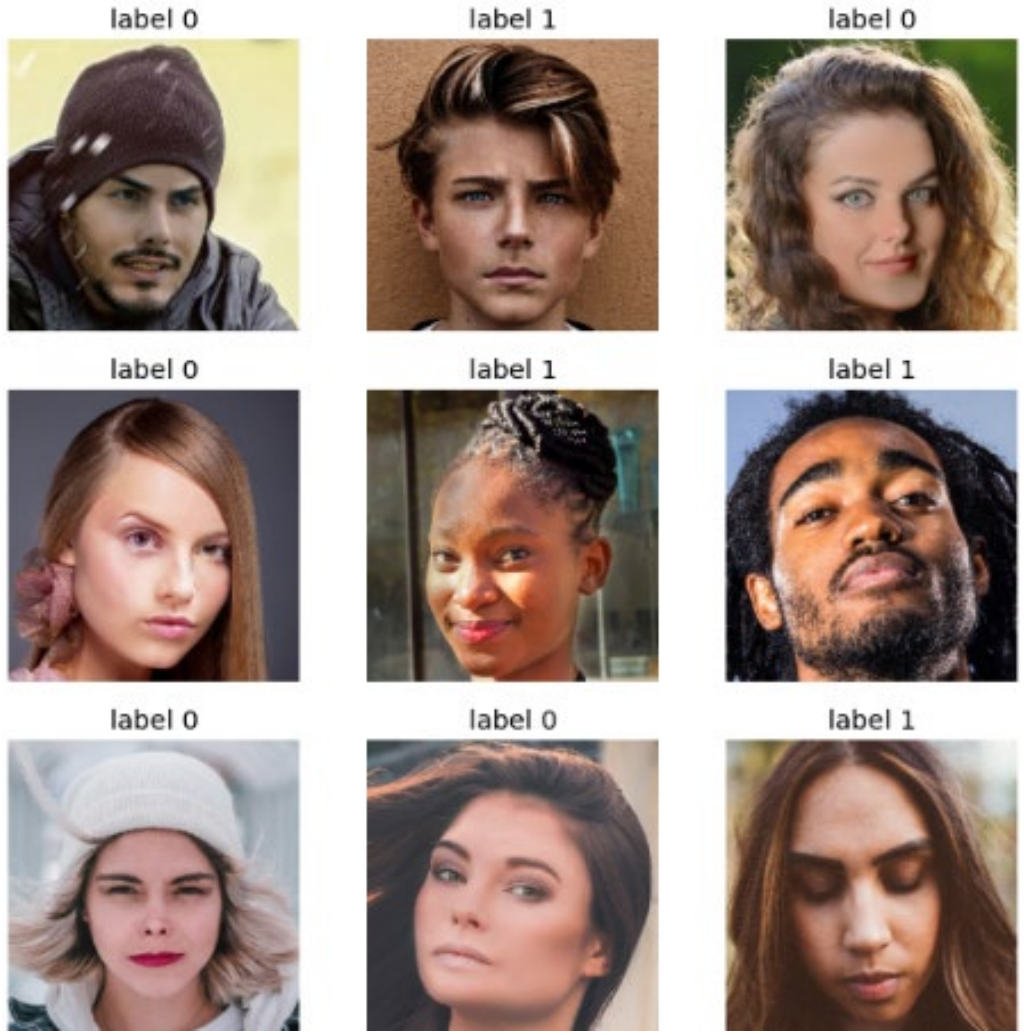
Transfer Learning y Fine Tuning en Keras

Entrenamiento y Evaluación

Entrenamiento

El entrenamiento y la evaluación se hacen como siempre.

```
tl_model.fit(x=X_train,
                      validation_data=...,
                      epochs=...,
                      callbacks=...)
```

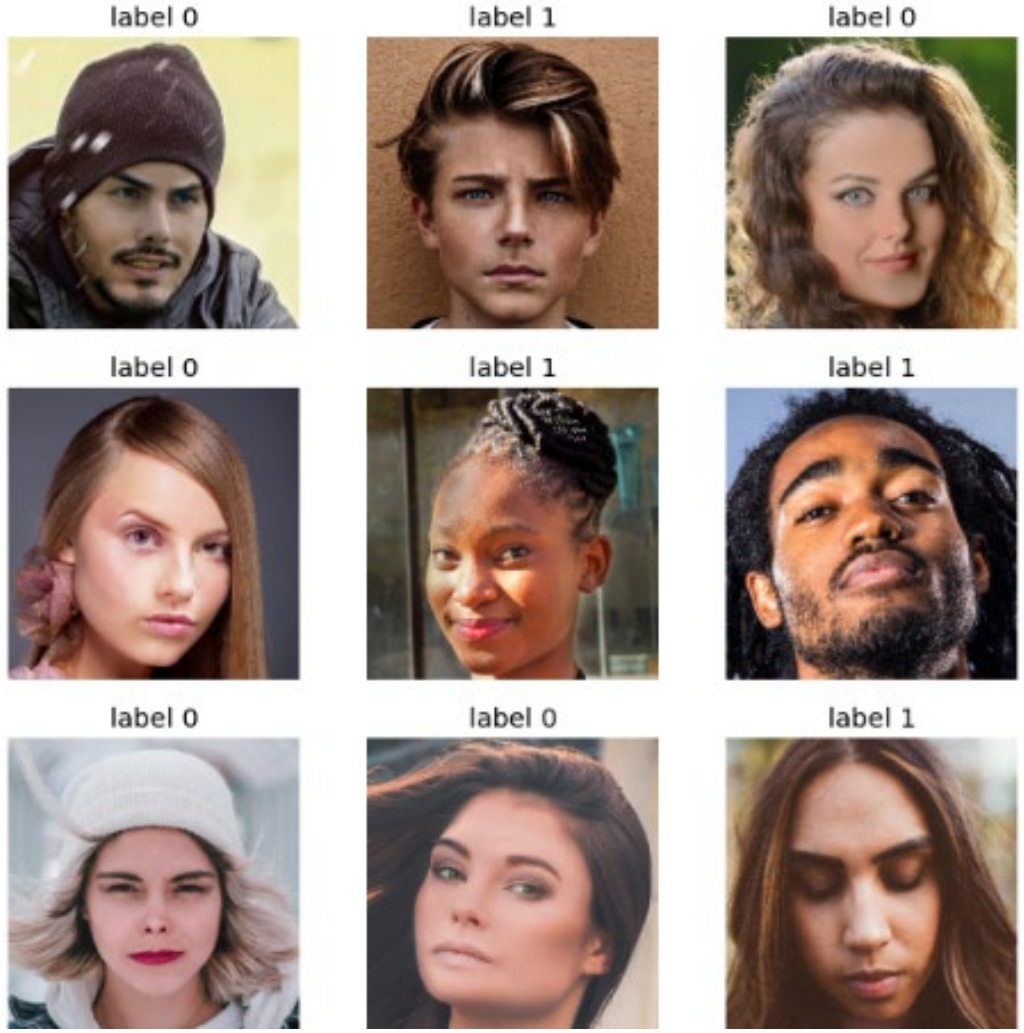


Transfer Learning y Fine Tuning en Keras

Entrenamiento y Evaluación

Resultados

	precision	recall	f1-score	support
0	0.62	0.54	0.58	284
1	0.64	0.71	0.68	329
accuracy			0.63	613
macro avg	0.63	0.63	0.63	613
weighted avg	0.63	0.63	0.63	613



Transfer Learning y Fine Tuning en Keras

Entrenamiento y Evaluación

Fine Tuning

```
extractor =
keras.applications.MobileNet(weights='imagenet',
                             include_top=False,
                             input_shape=(224, 224, 3))
```

Después de transfer learning, podemos hacer fine tuning.

Se cargan el modelo con los pesos del entrenamiento con ImageNet.

No se incluye en el modelo el clasificador final de 1000 clases.

Se define el tamaño de las imágenes de entrada.

```
for layer in extractor.layers:
    layer.trainable=False
```

El extractor se congela para el Warming Up.

```
pool = keras.layers.GlobalAveragePooling2D()(extractor.output)
dense1 = keras.layers.Dense(units=32, activation="relu")(pool)
drop1= keras.layers.Dropout(0.2)(dense1)
dense2 = keras.layers.Dense(units=1, activation="sigmoid")(drop1)

tl_model = keras.models.Model(inputs=[extractor.input], outputs=[dense2])
```

Agregamos capas densas nuevas.

Es un MLP.

La entrada son las features dadas por la CNN.

La salida es de dos clases.

Estas capas no están entrenadas.

Transfer Learning y Fine Tuning en Keras

Entrenamiento y Evaluación

El modelo se compila como siempre.

```
tl_model.compile(loss="binary_crossentropy",
    optimizer=keras.optimizers.Adam(lr=1e-3),
    metrics=["accuracy"])
```

Las capas añadidas se entrenan durante unas pocas epochs, antes de entrenar todo el modelo.

Si empezamos a entrenar todo, sin ajustar primero las capas nuevas, se van a **propagar errores** hacia atrás que van a dañar el modelo pre-entrenado con ImageNet.

```
ft_model.fit_generator(X_train,
                       validation_data=...,
                       epochs=2)
```

Se liberan las capas del extractor de características. Pueden liberarse todas, o un subconjunto.

```
for layer in ft_model.layers:
   layer.trainable = True
```

Transfer Learning y Fine Tuning en Keras

Entrenamiento y Evaluación

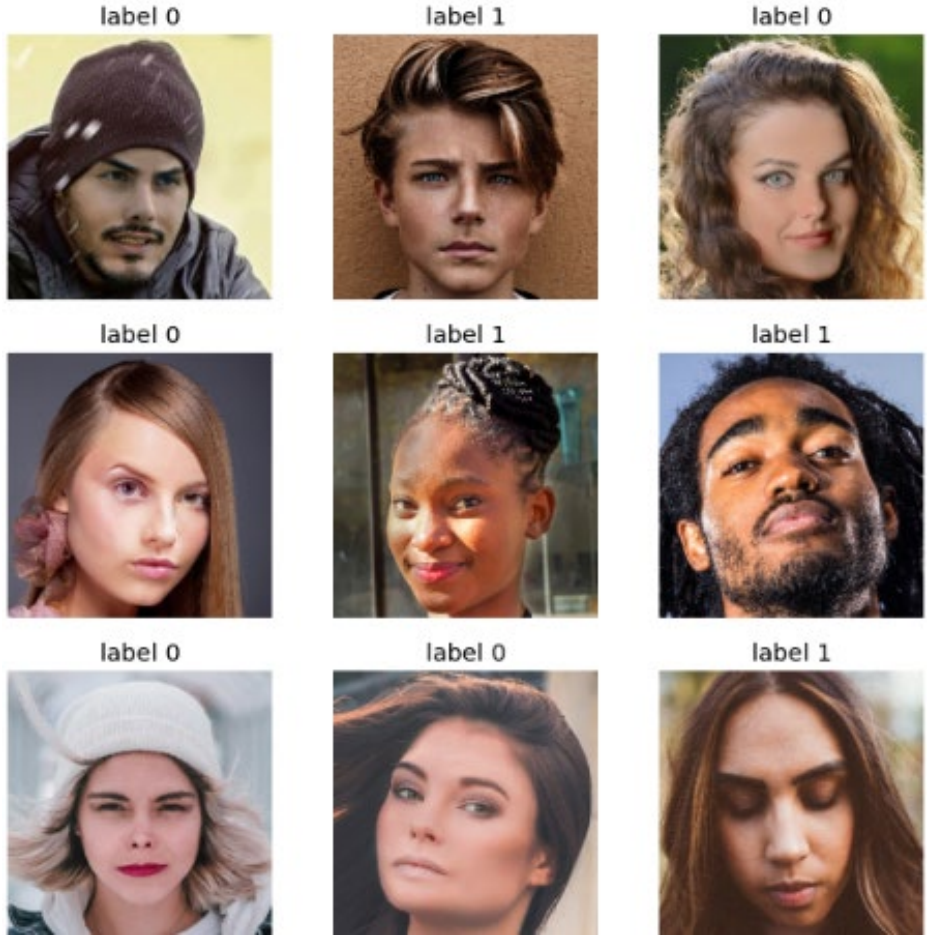
Compilación

```
tl_model.compile(loss="binary_crossentropy",
    optimizer=keras.optimizers.Adam(lr=1e-4),
    metrics=["accuracy"])
```

Se compila de nuevo con un learning rate más pequeño.

Transfer Learning y Fine Tuning en Keras

Entrenamiento y Evaluación



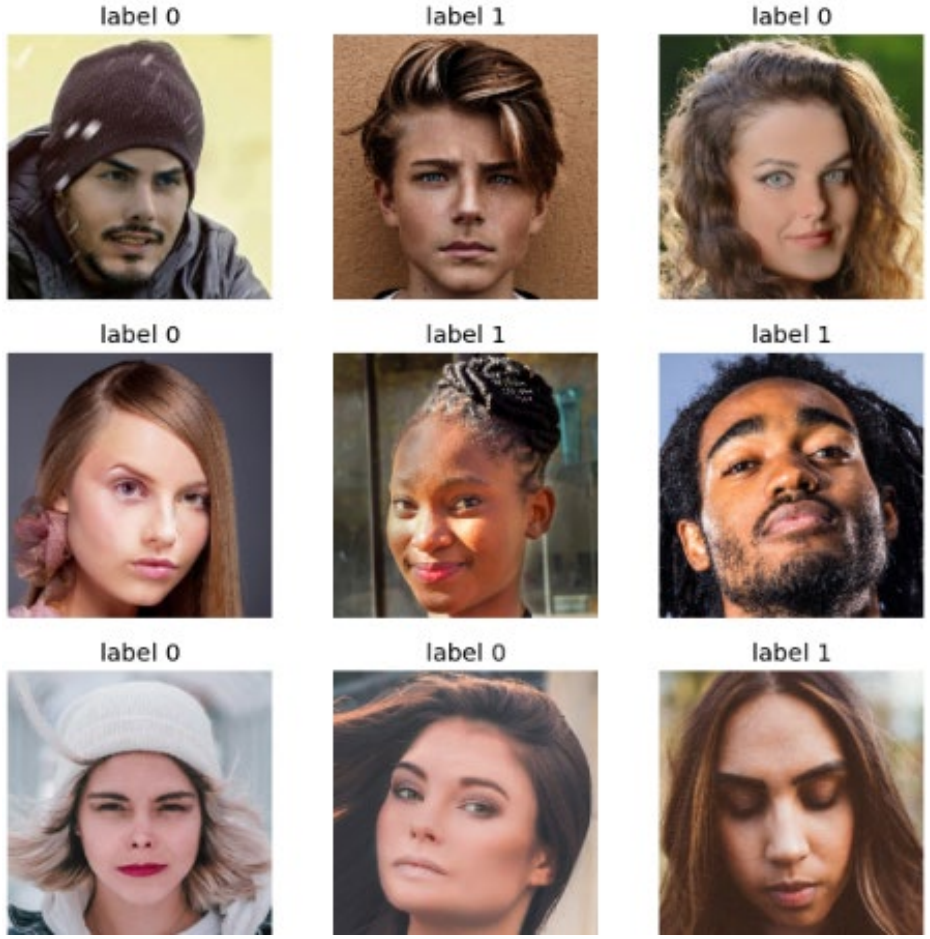
Entrenamiento

El entrenamiento y la evaluación se hacen como siempre.

```
tl_model.fit(x=X_train,
                        validation_data=...,
                        epochs=...,
                        callbacks=...)
```

Transfer Learning y Fine Tuning en Keras

Entrenamiento y Evaluación



Resultados

	precision	recall	f1-score	support
0	0.73	0.67	0.70	284
1	0.73	0.78	0.76	329
accuracy			0.73	613
macro avg	0.73	0.73	0.73	613
weighted avg	0.73	0.73	0.73	613

Transfer Learning y Fine Tuning en Keras

Entrenamiento y Evaluación

Transfer Learning

	precision	recall	f1-score
0	0.62	0.54	0.58
1	0.64	0.71	0.68
accuracy			0.63
macro avg	0.63	0.63	0.63
weighted avg	0.63	0.63	0.63

Fine Tuning

	precision	recall	f1-score
0	0.73	0.67	0.70
1	0.73	0.78	0.76
accuracy			0.73
macro avg	0.73	0.73	0.73
weighted avg	0.73	0.73	0.73

Despedida

¡Gracias por su atención!

Oscar Julián Perdomo Charry, PhD

ojperdomoc@unal.edu.co

Departamento de Ingeniería Eléctrica y Electrónica
Facultad de Ingeniería
Universidad Nacional de Colombia
Sede Bogotá

UNIVERSIDAD NACIONAL DE COLOMBIA

Referencias

- Wikipedia contributors. (2023, 13 enero). Kernel (image processing). Wikipedia. https://en.wikipedia.org/wiki/Kernel_(image_processing)

- Brownlee, J (2019, 6 Mayo) How to Visualize Filters and Feature Maps in Convolutional Neural Networks. Machine Learning Mastery. https://machinelearningmastery.com/how-to-visualize-filters-and-feature-maps-in-convolutional-neural-networks/

- Image classification. (s. f.). TensorFlow. https://www.tensorflow.org/tutorials/images/classification

- Kim, J (2021, 9 diciembre). Take a look inside the CNN. - . Medium. https://medium.com/@Joocheol_Kim/take-a-look-inside-the-cnn-eb65d0dfdf94
- Transfer Learning Introduction Tutorials & Notes | Machine Learning. (2018, 10 mayo). HackerEarth. https://www.hackerearth.com/practice/machine-learning/transfer-learning/transfer-learning-intro/tutorial/

- Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. ArXiv, abs/1704.04861.

- Fine-tuning guidelines [Imagen]. https://image.slidesharecdn.com/stcbwed1730mac205-161230131625/95/aws-reinvent-2016-deep-learning-at-cloud-scale-improving-video-discoverability-by-scaling-up-caffe-on-aws-mac205-28-638.jpg?cb=1483103838

Referencias

- Transfer Learning Introduction Tutorials & Notes | Machine Learning. (2018b, mayo 10). HackerEarth. https://www.hackerearth.com/practice/machine-learning/transfer-learning/transfer-learning-intro/tutorial/

- Von Csefalvay, C. (2021, 9 diciembre). Transfer learning: the dos and don'ts - Starschema Blog. Medium. https://medium.com/starschema-blog/transfer-learning-the-dos-and-donts-165729d66625

- Sharma,H (2019, 7 febrero) Transfer Learning : Approaches and Empirical Observations. Hackernoon. https://hackernoon.com/transfer-learning-approaches-and-empirical-observations-efeff9dfeca6

- Sarkar, D. (2022, 5 octubre). A Comprehensive Hands-on Guide to Transfer Learning with Real-World Applications in Deep Learning. Medium. https://towardsdatascience.com/a-comprehensive-hands-on-guide-to-transfer-learning-with-real-world-applications-in-deep-learning-212bf3b2f27a

Derechos de imágenes

- Flaticon. (s.f.). Clustering free icon. [Icono]. https://www.flaticon.com/free-icon/clustering_7440393
- What a Deep Neural Network thinks about your #selfie. (2015, October 25)[Imagen]. https://karpathy.github.io/assets/selfie/useful.jpg
- Martins,A. Niculae,V (2019) Lecture 4: Representation Learning and Convolutional Networks - Hierarchical Compositionality. [PDF] https://andre-martins.github.io/docs/dsl2019/lecture_04.pdf
- Kornyakov,K(s.f.). ACCELERATING COMPUTER VISION WITH OPENCV AND CUDA - HUMAN VISION [PDF] https://on-demand.gputechconf.com/gtc/2014/presentations/HANDS-ON-LAB-S4795-accelerating-computer-vision-opencv-cuda.pdf
- LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition - Architecture of LeNet-5 [PDF] https://www.researchgate.net/publication/2985446_Gradient-Based_Learning_Applied_to_Document_Recognition
- Flaticon. (s.f.). Mona Lisa free icon.[Icono]. https://www.flaticon.com/free-icon/mona-lisa_1911305
- Nadeem Qazi. The window incorporates the depth, but it only moves along two dimensions of the image [GIF]. https://miro.medium.com/max/738/1*Q7NXeOlDkm4xlNrNQOS67g.gif
- Bouvet. A Pooling example with a stride of 2 and a filter size of 2x2 [GIF]. https://www.bouvet.no/bouvet-deler/understanding-convolutional-neural-networks-part-1/_/attachment/inline/e60e56a6-8bcd-4b61-880d-7c621e2cb1d5:6595a68471ed37621734130ca2cb7997a1502a2b/Pooling.gif
- Wikipedia. Discrete 2D Convolution Animation [GIF].https://en.m.wikipedia.org/wiki/Convolution#/media/File%3A2D_Convolution_Animation.gif
- J. (2020, July 1). Redes Neuronales Convolucionales en Profundidad. DataSmarts Español. [GIF] https://i1.wp.com/datasmarts.net/es/wp-content/uploads/2018/11/convolution-schematic.gif?resize=300%2C219&ssl=1

Derechos de imágenes

- Flaticon. (s.f.). Shih Tzu free icon. [Icono]. https://www.flaticon.com/free-icon/shih-tzu_2926757
- Kaggle. What is Image Data Augmentation [Imagen]. https://www.kaggle.com/getting-started/190280
- Flaticon. (s.f.). Knowledge Transfer free icon. [Icono]. https://www.flaticon.com/free-icon/knowledge-transfer_8017785
- Flaticon. (s.f.). Machine Learning free icon. [Icono]. https://www.flaticon.com/free-icon/machine-learning_8024491
- Flaticon. (s.f.). Versus free icon. [Icono]. https://www.flaticon.com/free-icon/versus_7960356
- Flaticon. (s.f.). Knowledge Transfer free icon. [Icono]. https://www.flaticon.com/free-icon/knowledge-transfer_4592902
- Flaticon. Free Check Icon [Icono]. https://www.flaticon.com/free-icon//cheque_3699516
- Flaticon. Free Cross Icon [Icono]. https://www.flaticon.com/free-icon//cruzar_6711656
- Flaticon. Free New Car Icon [Icono]. https://www.flaticon.com/free-icon//carro-nuevo_5044540
- Flaticon. Free Cart Icon [Icono]. https://www.flaticon.com/free-icon//carro_1574481
- Alex Krizhevsky. The CIFAR-10 dataset [JPG]. https://www.cs.toronto.edu/~kriz/cifar.html
- MDPI. Ball chart reporting the Top-1 ImageNet-1k accuracy vs. computational complexity [JPG]. https://www.mdpi.com/electronics/electronics-11-00945/article_deploy/html/images/electronics-11-00945-g001-550.jpg
- Flaticon. (s.f.). Service free icon. [Icono]. https://www.flaticon.com/free-icon/service_6672041
- Figura 1. RM cerebral diagnóstica, preoperatoria. A) Imagen axial en T2. . . (n.d.). ResearchGate. https://www.researchgate.net/profile/Herrera-Juan-Manuel/publication/331751046/figure/fig2/AS:736528099704837@1552613391344/Figura-1-RM-cerebral-diagnostica-preoperatoria-A-Imagen-axial-en-T2-sin-contraste-B.png

Derechos de imágenes

- Flaticon. (s.f.). Knowledge Transfer free icon. [Icono]. https://www.flaticon.com/free-icon/knowledge-transfer_9304691
- Flaticon. (s.f.). Settings free icon. [Icono]. https://www.flaticon.com/free-icon/settings_597292
- Team, K. (s. f.). Keras: the Python deep learning API. [icono] https://keras.io/img/logo.png
- Flaticon. (s.f.). Brain Circuit free icon. [Icono]. https://www.flaticon.com/free-icon/brain-circuit_10229607

Créditos

Facultad de
INGENIERÍA

Profesor
Fabio Augusto González Osorio, PhD

Asistente docente
Santiago Toledo Cortés, PhD (C)

Coordinador de virtualización
Edder Hernández Forero, Ing

Diagramador PPT
Mario Andres Rodriguez Triana

Diseño gráfico
Clara Valeria Suárez
Caballero
Milton R. Pachón Pinzón

2023

